

UNIVERSIDADE TUIUTI DO PARANÁ

**JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN**

**FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO
ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS
COM SIMULAÇÃO VISUAL INTERATIVA**

**CURITIBA
2026**

**JOÃO THOMAZ VIEIRA
KLAUS SIEGFRIED BECKMANN**

**FERRAMENTA BASEADA EM LINGUAGEM DE DOMÍNIO
ESPECÍFICO PARA O APOIO AO ENSINO DE AUTÔMATOS FINITOS
COM SIMULAÇÃO VISUAL INTERATIVA**

Trabalho de Conclusão de Curso apresentado ao curso de Bacharelado em Ciência da Computação da Faculdade de Ciências Exatas e de Tecnologia da Universidade Tuiuti do Paraná, como requisito à obtenção ao grau de Bacharel.

Orientador: Prof. Diógenes Cogo Furlan

**CURITIBA
2026**

AGRADECIMENTOS

Agradecemos a Manoel Gomes, o eterno Caneta Azul, cujo meme nos acompanhou nos momentos mais difíceis desta jornada acadêmica. Sua obra, de profundidade imensurável, foi a inspiração diária que nos deu forças para continuar.

RESUMO

O ensino de autômatos finitos em cursos de Ciência da Computação enfrenta dificuldades relacionadas à abstração dos conceitos teóricos, uma vez que as ferramentas existentes adotam abordagens exclusivamente visuais, sem oferecer representação textual formal, versionamento ou diagnósticos de erro precisos. Este trabalho propõe o desenvolvimento de uma ferramenta educacional composta por uma Linguagem de Domínio Específico (DSL) textual para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), acompanhada de um compilador que realiza análise léxica, sintática e semântica, fornecendo mensagens de erro com indicação de localização e natureza do problema. A ferramenta integra ainda uma interface gráfica interativa que renderiza visualmente o autômato descrito e anima o processo de simulação passo a passo, destacando os estados e transições percorridos durante o reconhecimento de cadeias de entrada. A validação do protótipo é realizada por meio de testes com autômatos clássicos da literatura, incluindo casos com erros propositais para avaliar a qualidade dos diagnósticos gerados. Espera-se que a ferramenta contribua para a compreensão prática do funcionamento de autômatos finitos, conectando a teoria de autômatos à prática de compiladores em um único instrumento educacional.

Palavras-chave: Autômatos Finitos; Linguagem de Domínio Específico; Compiladores; Simulação Visual; Ensino de Computação.

LISTA DE FIGURAS

FIGURA 1 – Árvore de derivação da palavra <i>aabb</i> na gramática <i>G</i>	14
FIGURA 2 – AFD que reconhece cadeias contendo 01 como subcadeia	16
FIGURA 3 – AFN que reconhece cadeias terminadas em 01	18
FIGURA 4 – AFD construído a partir do AFN da Figura 3 pela construção de subconjuntos	20
FIGURA 5 – Fases de um compilador	28
FIGURA 6 – Interações entre o analisador léxico e o analisador sintático . . .	29
FIGURA 7 – Árvore de derivação da expressão $id + id * id$	31
FIGURA 8 – Árvore sintática abstrata da expressão $id + id * id$	36

LISTA DE TABELAS

TABELA 1 – Hierarquia de Chomsky	14
TABELA 2 – Tabela de transição do AFD que reconhece cadeias contendo 01 .	21
TABELA 3 – Comparação entre bibliotecas de <i>parser combinators</i> para Rust .	48

LISTA DE SIGLAS E ACRÔNIMOS

AFD	Autômato Finito Determinístico
AFN	Autômato Finito Não-Determinístico
AST	Árvore Sintática Abstrata
BNF	Forma de Backus-Naur
DSL	Linguagem de Domínio Específico
EBNF	Forma de Backus-Naur Estendida
LLVM	Low Level Virtual Machine
PEG	Gramática de Expressão de Análise

LISTA DE SÍMBOLOS

Σ	Alfabeto
Σ^*	Conjunto de todas as palavras sobre Σ
ε	Palavra vazia
$ w $	Comprimento da palavra w
$\emptyset$	Conjunto vazio
$\in$	Pertence a
$\notin$	Não pertence a
$\subseteq$	Subconjunto
$\cup$	União
$\cap$	Interseção
$\mathcal{P}(Q)$	Conjunto das partes de Q
L^*	Fecho de Kleene da linguagem L
$\rightarrow$	Regra de produção
$\Rightarrow$	Derivação direta
$\Rightarrow^*$	Derivação em zero ou mais passos
δ	Função de transição
$\hat{\delta}$	Função de transição estendida a cadeias
$L(M)$	Linguagem reconhecida pelo autômato M
$L(G)$	Linguagem gerada pela gramática G

SUMÁRIO

1	INTRODUÇÃO	9
2	AUTÔMATOS FINITOS E LINGUAGENS FORMAIS	11
2.1	CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS	11
2.2	GRAMÁTICAS FORMAIS	12
2.3	AUTÔMATO FINITO DETERMINÍSTICO	15
2.4	AUTÔMATO FINITO NÃO-DETERMINÍSTICO	16
2.5	EQUIVALÊNCIA ENTRE AFD E AFN	18
2.6	REPRESENTAÇÕES DE AUTÔMATOS	20
2.7	SIMULAÇÃO DE AUTÔMATOS	22
2.8	APLICAÇÕES DE AUTÔMATOS	24
3	COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO	26
3.1	CONCEITOS FUNDAMENTAIS DE COMPILADORES	26
3.2	ANÁLISE LÉXICA	29
3.3	ANÁLISE SINTÁTICA	31
3.4	ANÁLISE SEMÂNTICA	33
3.5	AST (ÁRVORE SINTÁTICA ABSTRATA)	35
3.6	LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL)	36
3.7	CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS)	38
3.8	DIAGNÓSTICO DE ERROS	40
3.9	DSL NO ENSINO	42
4	RUST COMO LINGUAGEM DE IMPLEMENTAÇÃO	44
4.1	VISÃO GERAL DA LINGUAGEM	44
4.2	SISTEMA DE TIPOS, <i>OWNERSHIP</i> E <i>BORROWING</i>	45
4.3	ECOSSISTEMA E GERENCIADOR DE PACOTES CARGO	46
4.4	<i>PARSER COMBINATORS</i> EM RUST	46
	REFERÊNCIAS	49

1 INTRODUÇÃO

O ensino de Teoria da Computação, em especial o conteúdo de autômatos finitos, representa um desafio recorrente nos cursos de Bacharelado em Ciência da Computação. Conceitos como estados, transições, alfabetos e reconhecimento de cadeias são abstratos por natureza, e sua compreensão prática depende fortemente da capacidade do estudante de visualizar e simular o comportamento das máquinas formais descritas teoricamente em sala de aula.

As ferramentas disponíveis atualmente para apoiar esse aprendizado, como o JFLAP (RODGER; FINLEY, 2006), adotam uma abordagem exclusivamente visual baseada em arrastar e soltar, na qual o usuário constrói o autômato posicionando estados e conectando transições com o *mouse*. Embora essa abordagem seja útil para ilustrar conceitos, ela apresenta limitações relevantes: a descrição do autômato não é textual, o que dificulta o versionamento e o compartilhamento dos modelos; não há validação formal da estrutura com mensagens de erro claras e localizadas; e o estudante que comete um erro na construção não recebe um diagnóstico preciso do problema.

Diante desse cenário, este trabalho propõe o desenvolvimento de uma ferramenta educacional que adota uma abordagem diferente. A solução consiste em uma Linguagem de Domínio Específico (DSL) textual para a descrição declarativa de Autômatos Finitos Determinísticos (AFD) e Não-Determinísticos (AFN), acompanhada de um compilador que realiza análise léxica, sintática e semântica, fornecendo mensagens de erro com indicação precisa da localização e da natureza do problema. A ferramenta integra ainda uma interface gráfica interativa que renderiza visualmente o autômato descrito e anima o processo de simulação passo a passo, destacando os estados e transições percorridos durante o reconhecimento de cadeias de entrada.

Um aspecto adicional de relevância é que a própria ferramenta, por ser construída com técnicas reais de compilação, funciona como um exemplo prático dos conceitos que o estudante está aprendendo, conectando a teoria de autômatos à prática de compiladores em um único instrumento educacional. A ausência de uma ferramenta que integre descrição textual formal, validação com diagnósticos claros e simulação visual interativa justifica o desenvolvimento deste trabalho, que busca preencher essa lacuna com uma solução unificada e pedagogicamente orientada.

O objetivo geral deste trabalho é desenvolver uma ferramenta educacional baseada em uma Linguagem de Domínio Específico para apoiar o ensino de autômatos finitos, com simulação visual interativa. Para atingir esse objetivo, definem-se os seguintes objetivos específicos:

- a) projetar uma DSL textual com sintaxe própria para a descrição declarativa de AFDs e AFNs, com a gramática formalizada em notação BNF/EBNF;

- b) implementar um compilador para a DSL com análise léxica, sintática e semântica, construindo uma Árvore Sintática Abstrata (AST) e fornecendo mensagens de erro com indicação de localização e natureza do problema;
- c) desenvolver um simulador capaz de processar cadeias de entrada sobre os autômatos descritos, determinando aceitação ou rejeição e registrando o caminho percorrido;
- d) construir uma interface gráfica interativa que renderize o autômato e anime a simulação passo a passo, permitindo ao usuário controlar a velocidade e navegar entre os passos;
- e) validar o protótipo por meio de testes com autômatos clássicos da literatura, incluindo casos com erros propositalmente para avaliar a qualidade dos diagnósticos gerados.

Este trabalho está organizado como segue. O Capítulo 2 apresenta os conceitos teóricos de autômatos finitos e linguagens formais. O Capítulo 3 aborda linguagens de domínio específico e as fases de compilação. O Capítulo 4 apresenta a linguagem Rust e justifica sua adoção como plataforma de implementação da ferramenta proposta. O Capítulo ?? discute as ferramentas relacionadas e suas limitações. O Capítulo ?? descreve o projeto da DSL, sua sintaxe e gramática. O Capítulo ?? detalha a implementação do compilador e da interface gráfica. O Capítulo ?? apresenta os testes e os resultados obtidos. Por fim, o Capítulo ?? apresenta as considerações finais e trabalhos futuros.

2 AUTÔMATOS FINITOS E LINGUAGENS FORMAIS

Este capítulo apresenta os fundamentos teóricos necessários à compreensão dos autômatos finitos e das linguagens regulares, formalismos que constituem o objeto central da ferramenta proposta neste trabalho. Inicia-se com os conceitos básicos de linguagens formais (Seção 2.1) e gramáticas (Seção 2.2), seguindo-se a formalização dos autômatos finitos determinísticos (Seção 2.3) e não-determinísticos (Seção 2.4). Por fim, demonstra-se a equivalência entre ambos os modelos (Seção 2.5), resultado que justifica o uso de qualquer um deles na descrição de linguagens regulares.

2.1 CONCEITOS FUNDAMENTAIS DE LINGUAGENS FORMAIS

O estudo dos autômatos finitos exige a compreensão prévia de algumas estruturas matemáticas elementares: conjuntos, alfabetos, palavras e linguagens. Esta seção apresenta tais conceitos de forma sistemática, fixando a notação e a terminologia adotadas nos capítulos subsequentes.

Um **conjunto** é uma coleção de elementos, que podem ser de qualquer natureza, como números, letras, símbolos ou mesmo outros conjuntos. Para indicar que um elemento a pertence a um conjunto C , escreve-se $a \in C$; o símbolo $\notin$ denota a relação inversa. Por exemplo, dado $C = \{a, b, c\}$, tem-se $a \in C$ e $d \notin C$. Conjuntos podem ser finitos ou infinitos; no segundo caso, é usual o emprego de reticências para indicar a continuação dos elementos, como em $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$. O conjunto que não contém qualquer elemento é denominado **conjunto vazio** e é representado pelo símbolo $\emptyset$ (SIPSER, 2012).

A partir desta noção de conjunto, define-se a primeira estrutura específica da teoria das linguagens formais: o alfabeto. Um **alfabeto** é um conjunto finito e não vazio de símbolos, representado, por convenção, pela letra grega Σ (SIPSER, 2012). Os elementos de um alfabeto são denominados símbolos ou letras. Por exemplo, $\Sigma_1 = \{0, 1\}$ é o alfabeto binário, enquanto $\Sigma_2 = \{a, b, c, \dots, z\}$ corresponde ao alfabeto latino minúsculo. A exigência de finitude é fundamental, pois garante que os autômatos, definidos nas seções seguintes, possam manipular o alfabeto de forma efetiva; a não vacuidade, por sua vez, assegura que sempre haja ao menos um símbolo disponível para a formação de palavras (VIEIRA, 2006; DIVERIO; MENEZES, 2011).

Sobre um alfabeto Σ , define-se uma **palavra** (ou cadeia) como uma sequência finita de símbolos pertencentes a Σ , escritos de forma justaposta e sem separadores. O comprimento de uma palavra w , denotado por $|w|$, corresponde ao número de símbolos que a compõem. A palavra de comprimento zero, denominada **palavra vazia**, é representada pelo símbolo ε (SIPSER, 2012). O conjunto de todas as palavras que podem ser formadas sobre Σ , incluindo a palavra vazia, é denotado por Σ^* . Assim, para

$\Sigma = \{a, b\}$, tem-se $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}$, conjunto que é sempre infinito e enumerável (VIEIRA, 2006).

Estabelecidas as noções de alfabeto e palavra, é possível definir o objeto central deste trabalho: a linguagem. Uma **linguagem formal** sobre um alfabeto Σ é qualquer subconjunto de Σ^* , ou seja, qualquer conjunto de palavras formadas a partir dos símbolos de Σ (SIPSER, 2012; VIEIRA, 2006). Linguagens podem ser finitas, como $L_1 = \{a, ab, abb\}$, ou infinitas, como $L_2 = \{a^n b^n \mid n \geq 1\}$, que contém todas as palavras compostas por uma sequência de a 's seguida de igual número de b 's. A maior parte das linguagens de interesse teórico e prático é infinita, o que motiva o desenvolvimento de mecanismos formais, como gramáticas e autômatos, capazes de descrevê-las de maneira finita.

Por serem conjuntos, as linguagens admitem as operações usuais de **união**, **interseção** e **diferença**. Além dessas, três operações específicas, derivadas da estrutura sequencial das palavras, desempenham papel central na teoria (VIEIRA, 2006; DIVERIO; MENEZES, 2011). A **concatenação** de duas palavras $x = a_1 a_2 \dots a_m$ e $y = b_1 b_2 \dots b_n$ produz a palavra $xy = a_1 a_2 \dots a_m b_1 b_2 \dots b_n$; em particular, $w\varepsilon = \varepsilon w = w$ para qualquer palavra w , o que faz de ε o elemento neutro dessa operação. A concatenação estende-se naturalmente a linguagens: dadas L_1 e L_2 , define-se $L_1 L_2 = \{xy \mid x \in L_1 \text{ e } y \in L_2\}$. Por fim, o **fecho de Kleene** de uma linguagem L , denotado por L^* , corresponde ao conjunto de todas as palavras obtidas pela concatenação de zero ou mais elementos de L , isto é, $L^* = \bigcup_{n \in \mathbb{N}} L^n$, onde $L^0 = \{\varepsilon\}$ e $L^n = L^{n-1} L$ para $n \geq 1$. Essas operações constituem a base para a especificação concisa de linguagens, recurso amplamente explorado nas expressões regulares e gramáticas estudadas nas seções seguintes.

2.2 GRAMÁTICAS FORMAIS

Conforme observado na seção anterior, a maior parte das linguagens de interesse prático é infinita, o que torna inviável descrevê-las pela enumeração explícita de suas palavras. Faz-se necessário, portanto, um mecanismo finito capaz de gerar todas as palavras de uma linguagem. Entre os formalismos propostos para esse fim, as **gramáticas** ocupam posição de destaque, tanto pelo seu poder expressivo quanto pela estreita relação com os autômatos estudados nas seções subsequentes (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998).

Uma gramática descreve uma linguagem por meio de um conjunto finito de **regras de produção**, que indicam como certas sequências de símbolos podem ser substituídas por outras. Nessa estrutura, distinguem-se dois tipos de símbolos: os **terminais**, que compõem o alfabeto da linguagem gerada e aparecem nas palavras finais, e os **não-terminais** (ou **variáveis**), símbolos auxiliares que servem de interme-

diários no processo de geração e não fazem parte das palavras produzidas (LEWIS; PAPADIMITRIOU, 1998; SIPSER, 2012). A geração de uma palavra inicia-se sempre a partir de uma variável especial, denominada **símbolo inicial** (ou variável de partida), e prossegue pela aplicação sucessiva das regras até que reste apenas uma sequência de terminais.

Formalmente, uma **gramática** é uma quádrupla $G = (V, \Sigma, R, S)$, na qual (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998):

- V é um conjunto finito de variáveis (símbolos não-terminais);
- Σ é um alfabeto de símbolos terminais, com $V \cap \Sigma = \emptyset$;
- R é um conjunto finito de **regras de produção**, cada uma escrita na forma $\alpha \rightarrow \beta$, em que $\alpha, \beta \in (V \cup \Sigma)^*$ e o lado esquerdo α contém ao menos uma variável;
- $S \in V$ é a variável de partida.

A aplicação de uma regra $\alpha \rightarrow \beta \in R$ a uma cadeia de variáveis e terminais produz uma nova cadeia pela substituição de uma ocorrência de α por β . Essa relação de substituição direta é denotada por $\Rightarrow$: dadas $u, v \in (V \cup \Sigma)^*$, escreve-se $u \Rightarrow v$ quando existem $x, y \in (V \cup \Sigma)^*$ e uma regra $\alpha \rightarrow \beta \in R$ tais que $u = x\alpha y$ e $v = x\beta y$ (LEWIS; PAPADIMITRIOU, 1998). Em outras palavras, x e y representam o trecho da cadeia que permanece inalterado, enquanto a subcadeia α , situada entre eles, é trocada por β . A título de exemplo, dada a regra $A \rightarrow ab$ e a cadeia cAd , tem-se $cAd \Rightarrow cabd$, identificando-se $x = c$, $\alpha = A$, $y = d$ e $\beta = ab$.

A aplicação sucessiva de regras, isto é, o fecho reflexivo e transitivo de $\Rightarrow$, é denotada por $\Rightarrow^*$. A **linguagem gerada** pela gramática G é o conjunto de todas as palavras formadas exclusivamente por terminais que podem ser obtidas a partir do símbolo inicial: $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ (VIEIRA, 2006; SIPSER, 2012).

A título de ilustração, considere a gramática $G = (\{S\}, \{a, b\}, R, S)$, em que $R = \{S \rightarrow aSb, S \rightarrow \varepsilon\}$. Partindo do símbolo inicial S , é possível obter, por exemplo, a palavra $aabb$ por meio da seguinte derivação:

$$\begin{array}{ll}
 S \Rightarrow aSb & \text{(aplicação da regra } S \rightarrow aSb\text{)} \\
 \Rightarrow aaSbb & \text{(aplicação da regra } S \rightarrow aSb\text{)} \\
 \Rightarrow aabb & \text{(aplicação da regra } S \rightarrow \varepsilon\text{)}
 \end{array}$$

A estrutura dessa derivação pode ser visualizada por meio de uma **árvore de derivação**, representada na Figura 1, na qual a raiz corresponde ao símbolo inicial, os nós internos representam as variáveis substituídas e as folhas correspondem aos terminais que compõem a palavra gerada (SIPSER, 2012).

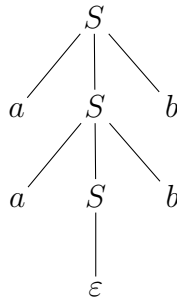


FIGURA 1 – Árvore de derivação da palavra *aabb* na gramática *G*

FONTE: Os autores (2026)

De modo geral, cada aplicação da regra $S \rightarrow aSb$ acrescenta um a à esquerda e um b à direita do símbolo S , enquanto a regra $S \rightarrow \varepsilon$ encerra a derivação eliminando a variável remanescente. Conclui-se que $L(G) = \{a^n b^n \mid n \geq 0\}$, linguagem que, conforme será discutido adiante, não pode ser gerada por nenhum mecanismo mais restrito que uma gramática livre de contexto (LEWIS; PAPADIMITRIOU, 1998; SIPSER, 2012).

Restrições impostas sobre a forma das regras de produção dão origem a diferentes classes de gramáticas e, conseqüentemente, a diferentes classes de linguagens. Essa estratificação, conhecida como **Hierarquia de Chomsky**, organiza as linguagens em quatro categorias (irrestritas, sensíveis ao contexto, livres de contexto e regulares), cada uma associada a um modelo computacional específico (VIEIRA, 2006; LEWIS; PAPADIMITRIOU, 1998). O Quadro 1 sintetiza essa classificação, evidenciando a forma das regras de produção admitidas em cada classe e o modelo computacional que reconhece a linguagem correspondente.

TABELA 1 – Hierarquia de Chomsky

Tipo	Classe de Linguagem	Forma das Regras	Modelo Computacional
0	Recursivamente enumeráveis	$\alpha \rightarrow \beta$	Máquina de Turing
1	Sensíveis ao contexto	$\alpha A \beta \rightarrow \alpha \gamma \beta$	Autômato linearmente limitado
2	Livres de contexto	$A \rightarrow \gamma$	Autômato com pilha
3	Regulares	$A \rightarrow aB$ ou $A \rightarrow a$	Autômato finito

FONTE: Adaptado de Vieira (2006) e Lewis e Papadimitriou (1998)

As gramáticas regulares (Tipo 3), em particular, geram exatamente a classe das linguagens reconhecidas pelos autômatos finitos, formalismo apresentado nas próximas seções.

2.3 AUTÔMATO FINITO DETERMINÍSTICO

Conforme indicado ao final da seção anterior, as gramáticas regulares (Tipo 3 da Hierarquia de Chomsky) descrevem exatamente a classe das linguagens reconhecidas pelo modelo computacional mais simples: o autômato finito. Esta seção apresenta a formalização do **autômato finito determinístico** (AFD), modelo que serve como ponto de partida para o estudo dos reconhecedores de linguagens regulares e que constitui o objeto central da ferramenta proposta neste trabalho.

Intuitivamente, um AFD é uma máquina abstrata composta por um número finito de **estados**, interligados por **transições** rotuladas com símbolos de um alfabeto. A máquina inicia sua operação em um estado especial, denominado **estado inicial**, e processa uma cadeia de entrada lendo seus símbolos um a um, da esquerda para a direita. A cada símbolo lido, o autômato transita para um novo estado, conforme determinado pela rotulação das arestas. Encerrado o processamento da cadeia, se o autômato encontrar-se em um dos estados designados como **estados finais** (ou de aceitação), diz-se que a cadeia foi **aceita**; caso contrário, é **rejeitada** (HOPCROFT *et al.*, 2006; SIPSER, 2012). O termo *determinístico* reflete a característica de que, para cada combinação de estado atual e símbolo de entrada, existe uma única transição possível.

Formalmente, um **autômato finito determinístico** é uma quintupla $M = (Q, \Sigma, \delta, q_0, F)$, na qual (HOPCROFT *et al.*, 2006; SIPSER, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \rightarrow Q$ é a **função de transição**, que associa a cada par formado por um estado e um símbolo o estado para o qual o autômato transita;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais (ou de aceitação).

A função δ descreve o comportamento do autômato ao ler um único símbolo. Para representar o processamento de uma cadeia inteira, define-se sua extensão recursiva $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, dada por (HOPCROFT *et al.*, 2006):

$$\begin{aligned}\hat{\delta}(q, \varepsilon) &= q, \\ \hat{\delta}(q, wa) &= \delta(\hat{\delta}(q, w), a),\end{aligned}$$

em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o estado em que o autômato se encontra após processar uma cadeia w a partir do estado q . A **linguagem reconhecida** (ou aceita) por um AFD M , denotada $L(M)$, é o conjunto de todas as

cadeias que conduzem o autômato, partindo do estado inicial, a algum estado de aceitação:

$$L(M) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \}.$$

As linguagens reconhecidas por algum AFD são denominadas **linguagens regulares** (SIPSER, 2012).

A representação gráfica de um AFD é feita por meio de um **diagrama de estados**, no qual os estados são representados por círculos, as transições por arestas rotuladas com os símbolos correspondentes, o estado inicial é apontado por uma seta sem origem e os estados de aceitação são marcados com círculos duplos. A título de exemplo, considere o AFD apresentado na Figura 2, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que contêm 01 como subcadeia (HOPCROFT *et al.*, 2006).

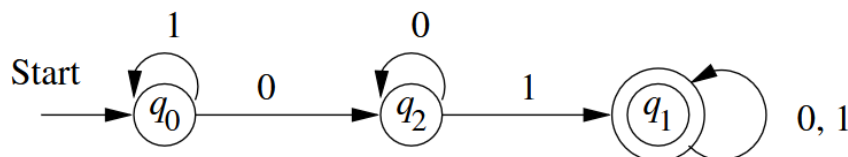


FIGURA 2 – AFD que reconhece cadeias contendo 01 como subcadeia

FONTE: Hopcroft *et al.* (2006)

Formalmente, o autômato da Figura 2 é descrito como $M = (Q, \Sigma, \delta, q_0, F)$, em que $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$, $F = \{q_1\}$ e a função δ é dada pelas transições $\delta(q_0, 0) = q_2$, $\delta(q_0, 1) = q_0$, $\delta(q_2, 0) = q_2$, $\delta(q_2, 1) = q_1$, $\delta(q_1, 0) = q_1$ e $\delta(q_1, 1) = q_1$. Cada estado codifica uma informação distinta sobre o histórico da leitura: q_0 indica que ainda não foi lido nenhum 0 (ou que o último símbolo lido foi 1), q_2 indica que o último símbolo lido foi 0 e q_1 indica que a subcadeia 01 já foi identificada (HOPCROFT *et al.*, 2006).

A título de ilustração, ao processar a cadeia 1101, o autômato realiza a sequência de transições $q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_2 \xrightarrow{1} q_1$, atingindo o estado de aceitação q_1 , de modo que a cadeia é aceita. Por outro lado, ao processar a cadeia 111, o autômato permanece em q_0 ao longo de toda a leitura e a cadeia é rejeitada por não terminar em estado final. Verifica-se que $L(M)$ corresponde exatamente ao conjunto das cadeias sobre $\{0, 1\}$ que contêm a subcadeia 01, em conformidade com a especificação informal apresentada.

2.4 AUTÔMATO FINITO NÃO-DETERMINÍSTICO

O autômato finito determinístico apresentado na seção anterior caracteriza-se por possuir, para cada combinação de estado e símbolo de entrada, uma única transição definida. Em diversas situações, no entanto, é mais natural descrever um reconhecedor permitindo que, em certos estados, mais de uma transição esteja disponível para

o mesmo símbolo, ou ainda que, em outros, não haja qualquer transição definida (HOPCROFT *et al.*, 2006; SIPSER, 2012). Tal flexibilidade dá origem ao **autômato finito não-determinístico** (AFN), modelo que, embora distinto em comportamento, reconhece a mesma classe de linguagens que os AFDs.

A diferença essencial em relação ao AFD reside na natureza da função de transição. Enquanto, em um AFD, a aplicação de δ a um par estado-símbolo retorna um único estado, em um AFN ela retorna um **conjunto** de estados, possivelmente vazio. Isso significa que, ao processar um símbolo a partir de um dado estado, o AFN pode prosseguir por qualquer um dos estados indicados (caso haja mais de um) ou travar (caso o conjunto seja vazio). Intuitivamente, imagina-se que o autômato investiga simultaneamente todas as escolhas possíveis: o caminho computacional que, ao final da leitura, atinge um estado de aceitação determina a aceitação da cadeia, mesmo que outros caminhos tenham travado ou terminado em estados não-finais (SIPSER, 2012).

Formalmente, um **autômato finito não-determinístico** é uma quintupla $N = (Q, \Sigma, \delta, q_0, F)$, na qual (HOPCROFT *et al.*, 2006; SIPSER, 2012):

- Q é um conjunto finito de estados;
- Σ é um alfabeto finito de símbolos de entrada;
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ é a **função de transição**, que associa a cada par estado-símbolo um subconjunto de Q , sendo $\mathcal{P}(Q)$ o conjunto das partes de Q ;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados finais.

A extensão da função de transição a cadeias, denotada $\hat{\delta}$, é definida recursivamente de modo análogo ao caso determinístico, levando em consideração que cada passo pode produzir múltiplos estados (HOPCROFT *et al.*, 2006):

$$\begin{aligned}\hat{\delta}(q, \varepsilon) &= \{q\}, \\ \hat{\delta}(q, wa) &= \bigcup_{p \in \hat{\delta}(q, w)} \delta(p, a),\end{aligned}$$

em que $w \in \Sigma^*$ e $a \in \Sigma$. A função estendida indica, portanto, o conjunto de todos os estados em que o autômato pode encontrar-se após processar a cadeia w a partir do estado q . A **linguagem aceita** por um AFN N , denotada $L(N)$, é o conjunto das cadeias para as quais existe ao menos um caminho computacional terminando em estado de aceitação:

$$L(N) = \{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset \}.$$

Em outras palavras, basta que uma das possíveis sequências de transições conduza a um estado final para que a cadeia seja aceita (HOPCROFT *et al.*, 2006; SIPSER, 2012).

A título de exemplo, considere o AFN apresentado na Figura 3, que reconhece a linguagem das cadeias sobre $\{0, 1\}$ que terminam em 01 (HOPCROFT *et al.*, 2006).

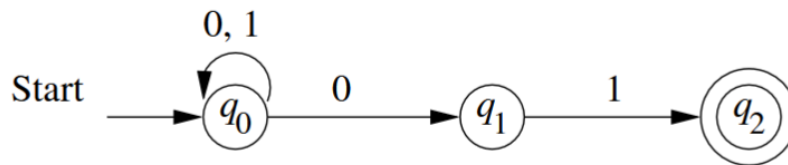


FIGURA 3 – AFN que reconhece cadeias terminadas em 01

FONTE: Hopcroft *et al.* (2006)

Esse autômato é descrito formalmente como $N = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$, em que a função de transição é dada por $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_2\}$ e os demais valores correspondem ao conjunto vazio. As duas características que evidenciam o não-determinismo ficam aparentes nessa especificação: o estado q_0 admite duas transições distintas para o símbolo 0 (o autômato pode permanecer em q_0 ou avançar para q_1), enquanto os estados q_1 e q_2 apresentam transições ausentes para certos símbolos.

Para ilustrar o funcionamento, considere o processamento da cadeia 001. Partindo de $\hat{\delta}(q_0, \varepsilon) = \{q_0\}$, calcula-se sucessivamente $\hat{\delta}(q_0, 0) = \{q_0, q_1\}$, $\hat{\delta}(q_0, 00) = \{q_0, q_1\}$ e $\hat{\delta}(q_0, 001) = \{q_0, q_2\}$. Como $q_2 \in F$, a cadeia é aceita. Por outro lado, ao processar a cadeia 010, obtém-se ao final $\hat{\delta}(q_0, 010) = \{q_0, q_1\}$, conjunto que não contém o estado final q_2 , de modo que a cadeia é rejeitada.

Embora o não-determinismo confira maior flexibilidade aparente na descrição de linguagens, demonstra-se que toda linguagem reconhecida por um AFN também pode ser reconhecida por algum AFD (HOPCROFT *et al.*, 2006; SIPSER, 2012). Esse resultado, fundamentado na chamada **construção de subconjuntos**, é apresentado na seção seguinte e justifica o fato de ambos os modelos definirem exatamente a mesma classe de linguagens: as linguagens regulares.

2.5 EQUIVALÊNCIA ENTRE AFD E AFN

Embora o autômato finito não-determinístico ofereça maior flexibilidade na descrição de linguagens, demonstra-se que essa flexibilidade não amplia a classe de linguagens reconhecíveis. Mais precisamente, vale o seguinte resultado central da teoria dos autômatos: uma linguagem L é reconhecida por algum AFD se, e somente se, é reconhecida por algum AFN (HOPCROFT *et al.*, 2006; SIPSER, 2012). Em outras

palavras, AFDs e AFNs são modelos **equivalentes em poder de reconhecimento**, ainda que possam diferir significativamente em número de estados e em facilidade de projeto.

A demonstração desse resultado é constituída por duas direções. A primeira, e mais imediata, consiste em observar que todo AFD pode ser interpretado como um AFN: basta tomar a função de transição do AFD e, para cada par estado-símbolo, considerar o conjunto unitário formado pelo único destino existente. Como o AFN resultante simula passo a passo o AFD original, ambos reconhecem a mesma linguagem (HOPCROFT *et al.*, 2006). A segunda direção, conhecida como **construção de subconjuntos** (*subset construction*), é menos trivial e fornece um procedimento algorítmico para converter qualquer AFN em um AFD equivalente.

A construção de subconjuntos parte da seguinte ideia: em um AFN, após ler uma cadeia w , o autômato pode encontrar-se em qualquer um dos estados pertencentes a $\hat{\delta}(q_0, w)$. O AFD equivalente, por sua vez, simula esse comportamento mantendo, em cada um de seus estados, o conjunto de estados do AFN em que a computação poderia estar a partir da leitura realizada até aquele momento. Os estados do AFD são, portanto, subconjuntos do conjunto de estados do AFN (HOPCROFT *et al.*, 2006).

Formalmente, dado um AFN $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, constrói-se o AFD equivalente $D = (Q_D, \Sigma, \delta_D, q_{0,D}, F_D)$ por meio das seguintes definições (HOPCROFT *et al.*, 2006):

- $Q_D = \mathcal{P}(Q_N)$, ou seja, cada estado do AFD é um subconjunto de estados do AFN;
- $q_{0,D} = \{q_0\}$, isto é, o estado inicial do AFD corresponde ao conjunto que contém apenas o estado inicial do AFN;
- $F_D = \{S \in Q_D \mid S \cap F_N \neq \emptyset\}$, ou seja, um subconjunto é considerado estado de aceitação do AFD se contiver ao menos um estado de aceitação do AFN;
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ para cada $S \in Q_D$ e cada $a \in \Sigma$, isto é, a transição do AFD a partir de um subconjunto S é a união das transições que o AFN executaria a partir de cada estado em S .

Embora $\mathcal{P}(Q_N)$ contenha $2^{|Q_N|}$ elementos no caso geral, em geral apenas uma pequena fração desses subconjuntos é efetivamente alcançável a partir do estado inicial. Na prática, o procedimento é aplicado de forma incremental: parte-se de $\{q_0\}$ e, a cada passo, computam-se as transições para os subconjuntos ainda não explorados, até que nenhum novo subconjunto seja gerado. A correção da construção é assegurada pelo seguinte teorema, demonstrado em Hopcroft *et al.* (2006): *uma linguagem $L \subseteq \Sigma^*$ é reconhecida por algum AFN se, e somente se, é reconhecida por algum AFD.*

A título de ilustração, considere novamente o AFN da Figura 3, que reconhece as cadeias sobre $\{0, 1\}$ terminadas em 01. Aplicando a construção de subconjuntos a esse autômato, parte-se do estado inicial $\{q_0\}$ e calculam-se as transições conforme o procedimento descrito. Tem-se, por exemplo, $\delta_D(\{q_0\}, 0) = \delta_N(q_0, 0) = \{q_0, q_1\}$ e $\delta_D(\{q_0\}, 1) = \{q_0\}$. Continuando o processo, geram-se sucessivamente os demais subconjuntos alcançáveis até que o AFD se estabilize. O resultado final é apresentado na Figura 4.

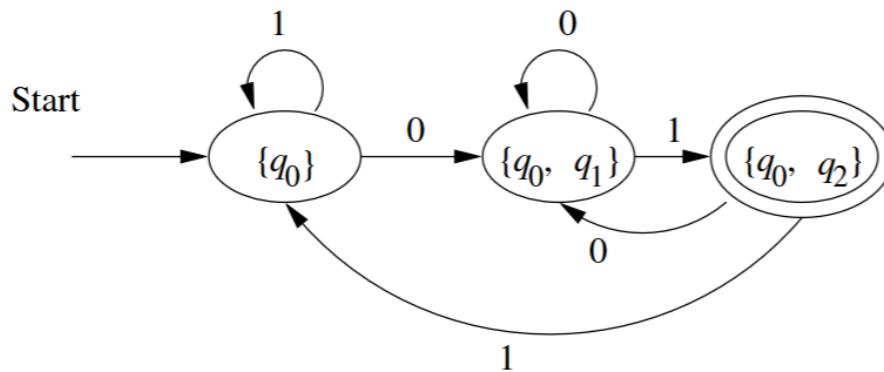


FIGURA 4 – AFD construído a partir do AFN da Figura 3 pela construção de subconjuntos

FONTE: Hopcroft *et al.* (2006)

Observa-se que o AFD resultante possui apenas três estados ($\{q_0\}$, $\{q_0, q_1\}$ e $\{q_0, q_2\}$), ainda que o conjunto das partes de Q_N contivesse oito subconjuntos possíveis. O estado $\{q_0, q_2\}$ é o único estado de aceitação, pois é o único subconjunto acessível que contém q_2 , estado final do AFN de origem. Verifica-se diretamente que $L(D) = L(N)$, isto é, o AFD construído reconhece exatamente a mesma linguagem do AFN de partida (HOPCROFT *et al.*, 2006).

Vale observar, por fim, que, no pior caso, a construção de subconjuntos pode produzir um AFD com 2^n estados a partir de um AFN com n estados, ainda que, na prática, o número de estados acessíveis costume ser comparável ao do AFN original (HOPCROFT *et al.*, 2006). Essa diferença, contudo, não altera a equivalência em poder expressivo: ambos os modelos definem exatamente a classe das linguagens regulares.

2.6 REPRESENTAÇÕES DE AUTÔMATOS

A definição formal de autômato finito como uma quintupla, apresentada nas seções anteriores, é suficiente para caracterizar matematicamente o objeto, mas não é necessariamente a forma mais prática de descrevê-lo no dia a dia. Na literatura e em ferramentas educacionais, são utilizadas três representações distintas e complementares, cada uma adequada a um propósito específico (HOPCROFT *et al.*, 2006; SIPSER,

2012). Esta seção apresenta essas representações, evidenciando as vantagens e limitações de cada uma.

A primeira forma é a **notação formal**, em que o autômato é descrito explicitamente como a quintupla $M = (Q, \Sigma, \delta, q_0, F)$, listando-se cada um de seus componentes. Trata-se da representação mais precisa e desambígua, sendo indispensável em demonstrações matemáticas e em provas de equivalência. Por outro lado, sua leitura direta é trabalhosa: a função de transição δ , em particular, costuma ser apresentada como uma enumeração de pares $\delta(q, a) = q'$, que rapidamente se torna extensa à medida que cresce o número de estados (HOPCROFT *et al.*, 2006).

A segunda forma é o **diagrama de estados**, já utilizado nas Figuras 2 e 3 das seções anteriores. Nessa representação, cada estado corresponde a um nó (círculo simples para estados comuns ou círculo duplo para estados de aceitação), as transições são arestas rotuladas com os símbolos do alfabeto, e o estado inicial é apontado por uma seta sem origem (HOPCROFT *et al.*, 2006; SIPSER, 2012). O diagrama de estados é o formato preferido em contextos didáticos e em ferramentas como o JFLAP (RODGER; FINLEY, 2006), pois permite que o leitor reconheça visualmente padrões como ciclos, estados de absorção e simetrias. Sua principal limitação é a dificuldade de manipulação textual: o diagrama é uma imagem e, portanto, não pode ser facilmente versionado, comparado ou processado por ferramentas automatizadas.

A terceira forma é a **tabela de transição**, que organiza a função δ em uma matriz bidimensional cujas linhas correspondem aos estados e cujas colunas correspondem aos símbolos do alfabeto (HOPCROFT *et al.*, 2006). A Tabela 2 ilustra essa representação para o AFD da Figura 2, que reconhece cadeias contendo 01 como subcadeia. A seta ($\rightarrow$) marca o estado inicial e o asterisco (*) destaca o estado de aceitação, conforme convenção adotada por Hopcroft *et al.* (2006).

TABELA 2 – Tabela de transição do AFD que reconhece cadeias contendo 01

	0	1
$\rightarrow q_0$	q_2	q_0
q_2	q_2	q_1
$*q_1$	q_1	q_1

FONTE: Adaptado de Hopcroft *et al.* (2006)

A leitura da tabela é direta: cada célula (q, a) contém o estado $\delta(q, a)$ para o qual o autômato transita ao ler o símbolo a a partir do estado q . Essa representação combina precisão formal e leitura sistemática, sendo compacta, permitindo verificar facilmente se a função de transição está completa (no caso do AFD, todas as células devem estar preenchidas) e sendo particularmente adequada para o processamento

programático, pois pode ser diretamente codificada como uma matriz ou estrutura de dados equivalente (HOPCROFT *et al.*, 2006; SIPSER, 2012). Para AFNs, a tabela de transição segue o mesmo formato, com a diferença de que cada célula contém um *conjunto* de estados em vez de um único estado, podendo, inclusive, conter o conjunto vazio quando não houver transição definida (HOPCROFT *et al.*, 2006).

As três representações apresentadas (notação formal, diagrama de estados e tabela de transição) são complementares e equivalentes entre si, descrevendo o mesmo objeto matemático sob perspectivas distintas. A escolha entre elas depende do contexto de uso: o rigor formal favorece a notação como quintupla; a comunicação didática favorece o diagrama; a manipulação computacional favorece a tabela. A ferramenta proposta neste trabalho explora essa equivalência ao oferecer uma quarta forma de representação, baseada em uma linguagem de domínio específico textual, que combina a precisão da notação formal com a estruturação tabular, permitindo simultaneamente a renderização do diagrama de estados e o processamento programático da descrição.

2.7 SIMULAÇÃO DE AUTÔMATOS

A definição formal de um autômato finito, vista nas seções anteriores, descreve sob a forma de quintupla os elementos que compõem a máquina, mas não esclarece, por si só, como verificar se uma cadeia específica é aceita ou rejeitada. Essa verificação, conhecida como **simulação**, consiste em executar passo a passo a função de transição δ a partir do estado inicial, consumindo cada símbolo da cadeia de entrada e observando se a configuração final corresponde a um estado de aceitação (HOPCROFT *et al.*, 2006; SIPSER, 2012). Em termos pedagógicos, a simulação cumpre um papel análogo ao da execução de um programa em uma linguagem de programação, pois permite que o estudante observe o comportamento do autômato projetado e confronte-o com o comportamento esperado (MORAZÁN; ANTUNEZ, 2014).

Morazán e Antunez (2014) argumentam que disciplinas introdutórias de linguagens formais, tradicionalmente, exigem que estudantes projetem máquinas e provem sua correção sem oferecer mecanismos para experimentar com os artefatos produzidos. Os autores defendem que essa ausência de retroalimentação imediata contraria a forma como o estudante de Ciência da Computação aprende a programar, em que compiladores e interpretadores fornecem mensagens de erro e advertências durante o desenvolvimento. A consequência observada é que estudantes inexperientes submetem com frequência soluções incorretas, mesmo para exercícios relativamente simples, e dedicam esforço a tentativas de prova de correção sobre projetos defeituosos (MORAZÁN; ANTUNEZ, 2014). Para mitigar esse problema, os autores propõem que a teoria da computação seja ensinada com ferramentas capazes de *construir* computações, isto é, ambientes em que a descrição formal possa ser traduzida em uma representação

executável e submetida a testes.

Formalmente, simular um AFD $M = (Q, \Sigma, \delta, q_0, F)$ sobre uma cadeia $w = a_1a_2 \dots a_n \in \Sigma^*$ corresponde a calcular a sequência de configurações $q_0, q_1, \dots, q_n$, em que $q_i = \delta(q_{i-1}, a_i)$ para $1 \leq i \leq n$. A cadeia w é aceita se, e somente se, $q_n \in F$ (HOPCROFT *et al.*, 2006). Para um AFD, a simulação é determinística e linear: cada símbolo lido conduz a exatamente um próximo estado, de modo que o custo computacional cresce proporcionalmente ao comprimento da cadeia. Essa propriedade torna o AFD particularmente adequado para implementações eficientes, pois pode ser realizado por uma estrutura de dados simples, como uma tabela de transição indexada pelo par (estado, símbolo) (SIPSER, 2012).

A simulação de um AFN, em contraste, exige tratamento adicional, pois a função de transição δ pode levar um mesmo par (estado, símbolo) a um conjunto de estados, e transições ε permitem mudança de estado sem consumir símbolos da entrada. Conforme descrito por Morazán e Antunez (2014) ao apresentar a função *apply-sm* da biblioteca FSM, a simulação de uma máquina não determinística é realizada por meio de uma busca em largura sobre todos os caminhos possíveis que o autômato pode seguir ao consumir a cadeia. Em cada passo, o simulador mantém o conjunto de estados alcançáveis e, ao final da entrada, verifica se algum desses estados pertence ao conjunto de aceitação. Alternativamente, conforme Hopcroft *et al.* (2006), o AFN pode ser convertido previamente em um AFD equivalente pelo algoritmo de construção de subconjuntos, transferindo a simulação para o caso determinístico ao custo de uma possível expansão exponencial do número de estados.

Além de produzir um veredito de aceitação ou rejeição, uma ferramenta de simulação útil para o ensino deve expor o *caminho percorrido* pelo autômato, isto é, a sequência de configurações intermediárias atravessadas durante o processamento da cadeia. Morazán e Antunez (2014) denominam essa funcionalidade *show-transitions* e destacam sua importância para que o estudante identifique exatamente em que ponto sua máquina diverge do comportamento esperado. Essa visualização do *trace* de execução substitui o exercício manual de acompanhar transições em papel, propenso a erros, por uma observação direta do comportamento da máquina, em consonância com a prática de depuração comum no desenvolvimento de *software*.

A relevância da simulação ganha ainda mais destaque quando se considera o fenômeno descrito por Morazán e Antunez (2014) de que estudantes, sem ambiente de teste, frequentemente partem para demonstrações formais sobre projetos incorretos, com perda de tempo e desmotivação como consequência. Esse argumento é central para o presente trabalho: a ferramenta proposta busca oferecer simulação visual interativa diretamente acoplada à descrição textual do autômato, de forma que o estudante possa, a partir de uma única especificação, examinar a estrutura do diagrama de estados, executar cadeias de teste e acompanhar a evolução das configurações,

recebendo retroalimentação imediata sobre o comportamento da máquina.

Cabe ainda diferenciar dois sentidos do termo simulação que aparecem na literatura. Em sentido estrito, simular um autômato é executar sua função de transição sobre uma entrada específica, como descrito acima. Em sentido mais amplo, ferramentas como o JFLAP (RODGER; FINLEY, 2006) e a biblioteca FSM (MORAZÁN; ANTUNEZ, 2014) oferecem também a possibilidade de *testar* um autômato com múltiplas cadeias geradas aleatoriamente, comparar duas máquinas quanto à equivalência de suas linguagens e visualizar transformações construtivas, como a conversão de AFN em AFD. Essas operações estendem a noção básica de simulação para um conjunto mais rico de recursos didáticos, ao qual se pode atribuir o termo *ambiente de experimentação*. A ferramenta proposta neste trabalho posiciona-se nesse espectro, oferecendo, em primeiro plano, a simulação visual passo a passo de autômatos finitos descritos por meio de uma linguagem de domínio específico, conforme será detalhado no Capítulo 3.

2.8 APLICAÇÕES DE AUTÔMATOS

Apesar de serem objetos teóricos, autômatos finitos não são uma abstração restrita ao ambiente acadêmico. Hopcroft *et al.* (2006) destacam que esse modelo computacional sustenta o funcionamento de diversas categorias de *software* e *hardware* utilizadas no dia a dia, sempre que um sistema precisa registrar, em um número finito de configurações, a parte relevante de seu histórico para decidir qual ação tomar diante da próxima entrada. Esta seção apresenta as principais áreas de aplicação descritas pelos autores e estabelece a ponte entre os conceitos formais discutidos até aqui e os capítulos seguintes deste trabalho.

Hopcroft *et al.* (2006) agrupam as aplicações dos autômatos finitos em quatro grandes categorias:

- **Projeto e verificação de circuitos digitais.** Sistemas digitais com um número finito de estados internos podem ser modelados diretamente como autômatos, permitindo que o comportamento do circuito seja descrito de forma rigorosa e verificado antes da implementação física. Cada estado do autômato corresponde a uma configuração possível do circuito, e cada transição corresponde à resposta do *hardware* a um sinal de entrada (HOPCROFT *et al.*, 2006).
- **Análise léxica de compiladores.** O componente responsável por dividir o texto-fonte em unidades léxicas (identificadores, palavras-chave, operadores e literais) é tipicamente implementado como um autômato finito. Hopcroft *et al.* (2006) ilustram esse uso com um autômato cuja função é reconhecer a palavra-chave `then`: cada estado corresponde a um prefixo da palavra (vazio, `t`, `th`, `the`, `then`), e a sequência de transições percorrida pela cadeia de entrada determina se o lexema

foi encontrado. Esse mesmo princípio se generaliza para o reconhecimento de qualquer conjunto finito de *tokens* descrito por expressões regulares.

- **Busca de padrões em grandes volumes de texto.** Sistemas que percorrem coleções extensas de documentos, páginas *web* ou arquivos em busca de palavras, expressões ou padrões mais complexos costumam compilar essas consultas em autômatos finitos, executando a varredura em tempo proporcional ao tamanho da entrada (HOPCROFT *et al.*, 2006). Utilitários clássicos do ambiente Unix, como *grep*, e bibliotecas modernas de expressões regulares apoiam-se diretamente nessa equivalência entre expressões regulares e autômatos finitos.
- **Verificação de sistemas com número finito de estados.** Protocolos de comunicação, protocolos criptográficos e outros sistemas reativos podem ser modelados como autômatos para que propriedades como ausência de *deadlock*, alcançabilidade de estados ou conformidade com uma especificação sejam verificadas automaticamente (HOPCROFT *et al.*, 2006).

A presença pervasiva dos autômatos finitos nessas áreas decorre de uma característica central do modelo: o número fixo de estados permite que sua representação seja implementada com uma quantidade limitada de recursos, seja como um circuito de *hardware*, seja como uma estrutura de dados em memória (HOPCROFT *et al.*, 2006). Esse equilíbrio entre simplicidade e poder expressivo explica por que o modelo permanece relevante mesmo após décadas de evolução das ferramentas de *software*.

Para o presente trabalho, a aplicação de maior interesse é a análise léxica, pois constitui o elo direto entre a teoria dos autômatos finitos e a construção de compiladores e interpretadores para linguagens de programação. A linguagem de domínio específico proposta nesta monografia, descrita no Capítulo 3, será analisada lexicamente por um autômato finito gerado a partir das regras léxicas da própria linguagem, em uma aplicação concreta dos conceitos apresentados neste capítulo.

3 COMPILADORES E LINGUAGENS DE DOMÍNIO ESPECÍFICO

Este capítulo apresenta os fundamentos teóricos de compiladores e linguagens de domínio específico que embasam a ferramenta proposta. Inicia-se com os conceitos gerais de compiladores e sua distinção em relação a interpretadores (Seção 3.1), seguindo-se a descrição detalhada das três fases do *front-end*: análise léxica (Seção 3.2), análise sintática (Seção 3.3) e análise semântica (Seção 3.4). Em seguida, apresenta-se a Árvore Sintática Abstrata como estrutura intermediária central do processo de compilação (Seção 3.5). A partir desses fundamentos, introduz-se o conceito de Linguagem de Domínio Específico (Seção 3.6), discutem-se as principais ferramentas e abordagens para sua construção (Seção 3.7), trata-se do diagnóstico de erros como componente essencial de qualidade (Seção 3.8) e, por fim, examina-se o uso de DSLs no contexto do ensino de linguagens formais (Seção 3.9), perspectiva que motiva diretamente o desenvolvimento do presente trabalho.

3.1 CONCEITOS FUNDAMENTAIS DE COMPILADORES

Antes de tratar das fases específicas de análise que utilizam autômatos finitos, é necessário situar o que é um compilador e qual o seu papel no processo de tradução de programas. Esta seção apresenta a definição clássica de compilador, distingue-o do interpretador, que é o outro tipo mais comum de processador de linguagem, e descreve, em alto nível, a sequência de fases que constituem o pipeline de compilação. Os conceitos aqui introduzidos servirão de apoio às seções seguintes, que detalham cada fase do *front-end* de um compilador.

Um **compilador** é um programa que recebe como entrada um texto escrito em uma linguagem de programação, denominada **linguagem fonte**, e produz como saída um programa equivalente em outra linguagem, denominada **linguagem objeto** (ou linguagem alvo) (AHO *et al.*, 2008). A equivalência exigida nesse processo é semântica: o programa objeto deve produzir, sobre as mesmas entradas, exatamente os mesmos resultados que o programa fonte produziria, ainda que o faça em uma linguagem de nível mais baixo, em geral próxima da arquitetura da máquina de execução. Além de traduzir o programa, o compilador desempenha um segundo papel essencial, o de relatar, com precisão, eventuais erros detectados durante a tradução, fornecendo ao programador o diagnóstico necessário para a correção do código fonte (AHO *et al.*, 2008).

A linguagem fonte costuma ser uma linguagem de alto nível, projetada para ser lida e escrita por humanos, ao passo que a linguagem objeto pode ser código de máquina diretamente executável, código de uma máquina virtual (como os *bytecodes* da JVM) ou ainda outra linguagem de alto nível, configurando-se nesse caso um *source-to-*

source compiler, ou tradutor de fonte para fonte (AHO *et al.*, 2008). Independentemente da forma da saída, o cerne da tarefa permanece o mesmo, isto é, realizar uma tradução fiel preservando o significado original.

O compilador não é a única forma de processador de linguagem. Um **interpretador**, em vez de produzir um programa objeto que será posteriormente executado, realiza diretamente, sobre as entradas fornecidas pelo usuário, as operações descritas no programa fonte (AHO *et al.*, 2008). A diferença entre os dois modelos pode ser sintetizada pelo papel do programa fonte em tempo de execução: para o compilador, ele é apenas insumo de uma fase prévia, ausente no momento em que o programa objeto roda; para o interpretador, ele permanece presente durante toda a execução, sendo lido e processado instrução a instrução.

Cada abordagem apresenta vantagens distintas. O programa objeto produzido por um compilador costuma ser substancialmente mais rápido na execução, pois a tradução para a linguagem da máquina é feita uma única vez e o resultado é executado diretamente pelo processador. Por outro lado, o interpretador costuma oferecer um melhor diagnóstico de erros em tempo de execução, uma vez que tem acesso direto à estrutura do programa fonte enquanto o avalia (AHO *et al.*, 2008). As duas estratégias não são mutuamente exclusivas, e muitas implementações modernas combinam-nas em arranjos híbridos. O processador de Java, por exemplo, primeiro compila o programa fonte para uma forma intermediária portátil (os *bytecodes*), que é então interpretada por uma máquina virtual; certas máquinas virtuais ainda recompilam dinamicamente trechos do código para a linguagem da máquina hospedeira, técnica conhecida como compilação *just-in-time*, que busca obter simultaneamente a portabilidade da interpretação e o desempenho da compilação (AHO *et al.*, 2008).

Internamente, um compilador está longe de ser uma caixa-preta monolítica. Aho *et al.* (AHO *et al.*, 2008) descrevem o processo de compilação como uma sequência de fases, cada uma transformando uma representação do programa fonte em outra, mais próxima do programa objeto, conforme ilustra a Figura 5. Essas fases agrupam-se em duas grandes etapas: a **análise**, responsável por compreender a estrutura e o significado do programa fonte, e a **síntese**, responsável por construir o programa objeto a partir dessa compreensão. A parte de análise é também denominada *front-end* do compilador, enquanto a parte de síntese constitui o *back-end*.

A etapa de análise costuma ser subdividida em três fases. A **análise léxica** (também chamada de *scanning* ou leitura) percorre o fluxo de caracteres do programa fonte e os agrupa em sequências significativas, denominadas lexemas; para cada lexema, produz um *token*, isto é, um par formado por um nome simbólico e, eventualmente, um valor de atributo associado. A **análise sintática** recebe o fluxo de *tokens* produzido pela fase léxica e o organiza em uma estrutura hierárquica, em geral uma árvore de sintaxe, que torna explícita a estrutura gramatical do programa. Por fim, a **análise semântica**

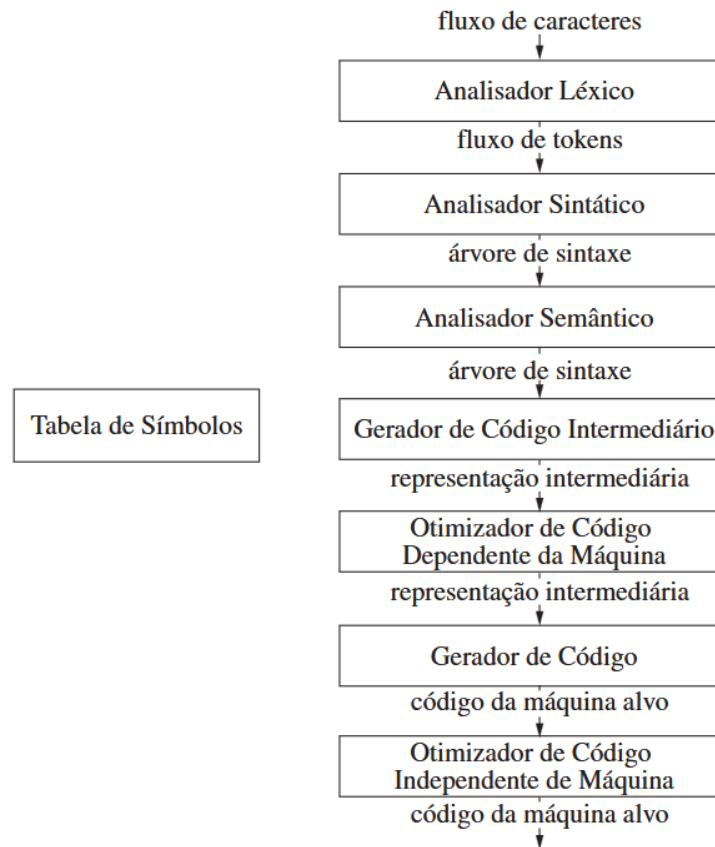


FIGURA 5 – Fases de um compilador

FONTE: Aho et al. (2008)

percorre essa árvore para verificar a consistência de tipos e o cumprimento de outras regras da linguagem que não podem ser expressas pela gramática livre de contexto, anotando a árvore com informações adicionais necessárias às fases seguintes (AHO *et al.*, 2008). Ao longo de todas essas fases, uma estrutura auxiliar denominada **tabela de símbolos** mantém informações sobre os identificadores declarados no programa, sendo consultada e atualizada conforme necessário.

Concluída a análise, a etapa de síntese gera, a partir da árvore anotada, uma **representação intermediária**, a qual pode ser submetida a uma fase opcional de otimização independente da máquina, com o objetivo de produzir código mais eficiente sem alterar a semântica do programa. Em seguida, o gerador de código traduz a representação intermediária para a linguagem da máquina alvo, e uma segunda fase opcional de otimização, dependente da máquina, refina o resultado explorando características específicas da arquitetura (AHO *et al.*, 2008). O programa objeto final é, portanto, produto de uma cadeia de transformações sucessivas, em que cada etapa preserva a semântica do programa original e adiciona ou remove informações conforme necessário.

Para os fins deste trabalho, interessa particularmente o *front-end* do compilador e, dentro dele, a fase de análise léxica, que constitui a aplicação direta dos autômatos

finitos discutidos no Capítulo 2. Os mecanismos pelos quais expressões regulares são convertidas em autômatos para o reconhecimento de *tokens*, bem como as estratégias de implementação desses autômatos em código, são o tema da seção seguinte.

3.2 ANÁLISE LÉXICA

A análise léxica é a primeira fase do *front-end* de um compilador e tem como tarefa principal ler o fluxo bruto de caracteres do programa fonte, agrupá-lo em sequências significativas, denominadas lexemas, e produzir, para cada lexema, um *token* que será consumido pela fase seguinte de análise sintática (AHO *et al.*, 2008). Além desse papel central, o analisador léxico costuma ser responsável por tarefas auxiliares de pré-processamento, como a remoção de espaços em branco e comentários, o registro do número da linha em que cada lexema ocorre (informação essencial para a posterior emissão de mensagens de erro) e, em algumas implementações, a expansão de macros (AHO *et al.*, 2008).

A interação entre os dois primeiros analisadores costuma ser implementada de forma orientada à demanda, conforme ilustra a Figura 6. O analisador sintático invoca o analisador léxico por meio de uma chamada normalmente denominada *getNextToken*, e este, por sua vez, lê caracteres da entrada até identificar o próximo lexema, sobre o qual constrói o *token* a ser devolvido. Esse arranjo permite que o analisador léxico interaja também com a tabela de símbolos: quando reconhece um lexema correspondente a um identificador, insere ou consulta a entrada apropriada nessa tabela, de modo que as fases subsequentes tenham acesso uniforme às informações sobre os identificadores do programa (AHO *et al.*, 2008).

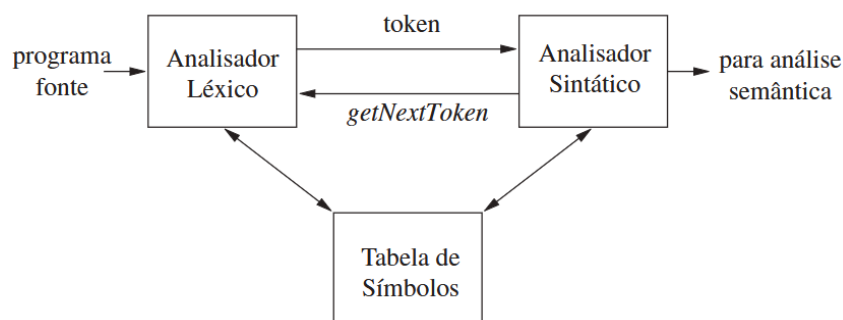


FIGURA 6 – Interações entre o analisador léxico e o analisador sintático

FONTE: Aho et al. (2008)

A separação entre análise léxica e análise sintática, embora pudesse, em princípio, ser dispensada, é justificada por três razões práticas, conforme apresenta Aho et al. (AHO *et al.*, 2008). A primeira é a simplicidade de projeto: tratar comentários e espaços em branco já no nível léxico evita poluir a gramática da fase sintática com construções acessórias. A segunda é a eficiência, pois técnicas especializadas de leitura, como

o uso de *buffers* duplos com sentinelas, podem ser aplicadas exclusivamente à fase léxica, acelerando substancialmente o processamento. A terceira é a portabilidade, uma vez que peculiaridades do dispositivo de entrada e da codificação de caracteres ficam restritas ao analisador léxico, sem afetar as demais fases do compilador.

A discussão sobre análise léxica apoia-se em três termos relacionados, porém distintos. Um **token** é um par formado por um nome simbólico e, opcionalmente, um valor de atributo; o nome representa, de forma abstrata, uma classe de unidade léxica, como uma palavra-chave específica ou a categoria genérica de identificador. Um **padrão** é uma descrição da forma que os lexemas associados a um *token* podem assumir, em geral expressa por meio de uma expressão regular. Um **lexema** é uma sequência concreta de caracteres do programa fonte que casa com o padrão de um *token* e é, por essa razão, classificada como uma instância desse *token* (AHO *et al.*, 2008). Por exemplo, em um programa em C que contenha o trecho `score = 3.14;`, a palavra `score` é um lexema que casa com o padrão de identificador e produz um *token id*, ao passo que `3.14` é um lexema que casa com o padrão de constante numérica e produz um *token number*.

Quando mais de um lexema pode casar com o mesmo padrão, como ocorre tipicamente com identificadores e constantes, o nome do *token* sozinho não basta para distinguir as ocorrências, e o analisador léxico precisa fornecer informação adicional às fases seguintes. Para isso, associa-se ao *token* um **valor de atributo**, que descreve o lexema específico encontrado. No caso de identificadores, esse atributo costuma ser um apontador para a entrada correspondente na tabela de símbolos, onde se mantêm informações como o nome, o tipo e a posição do identificador no programa fonte; no caso de constantes numéricas, o atributo pode ser o próprio valor numérico ou um apontador para uma representação interna desse valor (AHO *et al.*, 2008).

Eventualmente, o analisador léxico pode encontrar uma sequência de caracteres que não casa com nenhum dos padrões definidos. Em alguns casos, o erro só pode ser detectado por fases posteriores, pois o lexema é localmente válido, embora gramaticalmente inadequado: a sequência `fi`, em um programa C, é um identificador léxicamente válido, ainda que provavelmente seja um `if` grafado de forma equivocada (AHO *et al.*, 2008). Quando, porém, nenhum padrão casa com o prefixo restante da entrada, o analisador léxico precisa recorrer a alguma estratégia de recuperação. A mais simples, conhecida como recuperação em *modo de pânico*, consiste em descartar caracteres da entrada até que um *token* bem formado possa ser reconhecido; estratégias mais elaboradas tentam reparar o erro por meio de uma única transformação local (remoção, inserção, substituição ou transposição de caracteres), partindo da hipótese, empírica, de que a maioria dos erros léxicos envolve um único caractere (AHO *et al.*, 2008).

Do ponto de vista de implementação, a especificação dos padrões dos *tokens* costuma ser feita por meio de expressões regulares, e o reconhecimento, por meio de

autômatos finitos construídos a partir delas. Essa relação entre expressões regulares e autômatos, estabelecida no Capítulo 2, é exatamente o que torna a análise léxica uma aplicação direta da teoria de linguagens regulares: cada padrão corresponde a uma expressão regular, o conjunto desses padrões é convertido em um autômato finito não determinístico (em geral pelo método de Thompson), o qual é então transformado em um autômato finito determinístico (pela construção de subconjuntos) capaz de reconhecer eficientemente os lexemas durante a varredura do programa fonte (AHO *et al.*, 2008). Essa cadeia, de *regex* a *token*, fornece a base teórica para os geradores automáticos de analisadores léxicos, como o Lex e o Flex, e também para bibliotecas modernas que empregam técnicas equivalentes em linguagens contemporâneas. A ferramenta proposta neste trabalho, descrita no Capítulo 3, fará uso direto dessa correspondência ao analisar lexicamente sua linguagem de domínio específico, em uma aplicação concreta dos conceitos discutidos nesta seção.

3.3 ANÁLISE SINTÁTICA

A análise sintática, também denominada *parsing*, é a segunda fase do *front-end* de um compilador e tem por finalidade verificar se a sequência de *tokens* produzida pela análise léxica forma uma cadeia válida segundo a gramática da linguagem fonte e, em caso afirmativo, organizá-la em uma estrutura hierárquica que torne explícita a estrutura gramatical do programa (AHO *et al.*, 2008). Essa estrutura é, em geral, uma **árvore de derivação** (também chamada de *parse tree*), na qual os nós internos representam as variáveis aplicadas durante a derivação e as folhas correspondem aos terminais que compõem a cadeia analisada. A Figura 7 ilustra esse conceito para a expressão $id + id * id$, segundo a gramática $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$ e $F \rightarrow (E) \mid id$, tradicionalmente utilizada em livros-texto sobre análise sintática (AHO *et al.*, 2008). Observe que a estrutura hierárquica captura naturalmente a precedência dos operadores, pois a multiplicação, mais profunda na árvore, é avaliada antes da adição.

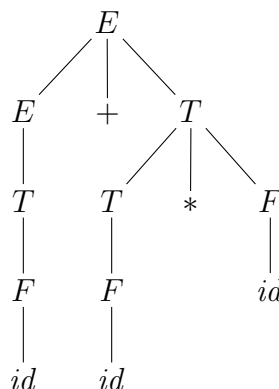


FIGURA 7 – Árvore de derivação da expressão $id + id * id$

FONTE: Os autores (2026)

A especificação da sintaxe das linguagens de programação utiliza, em geral, gramáticas livres de contexto, formalismo apresentado no Capítulo 2. A motivação dessa escolha é direta: as gramáticas regulares, embora suficientes para descrever os padrões léxicos, não conseguem expressar construções recursivamente aninhadas, como expressões parentizadas ou blocos de comandos balanceados, que são comuns em linguagens de programação. As gramáticas livres de contexto, por sua vez, capturam naturalmente essas construções por meio de regras recursivas, sendo o formalismo padrão para a especificação sintática de linguagens (AHO *et al.*, 2008).

Para programas bem formados, o analisador sintático constrói, ao menos conceitualmente, uma árvore de derivação que é então repassada às fases subsequentes do *front-end*, isto é, à análise semântica e à geração de código intermediário. Na prática, contudo, é comum que essa árvore não seja construída de forma explícita: as ações de tradução podem ser entremeadas com a análise propriamente dita, técnica conhecida como **tradução dirigida por sintaxe**, em que cada produção da gramática associa-se a um conjunto de ações semânticas executadas no momento em que a produção é reconhecida (AHO *et al.*, 2008). Esse arranjo permite, em muitos casos, integrar análise sintática, análise semântica e geração de código em um único módulo coeso.

Existem duas grandes famílias de algoritmos de análise sintática empregados em compiladores reais: a análise **descendente** (*top-down*) e a análise **ascendente** (*bottom-up*) (AHO *et al.*, 2008). Os métodos descendentes constroem a árvore de derivação da raiz para as folhas, partindo do símbolo inicial da gramática e tentando reescrevê-lo até obter a cadeia da entrada; nessa família, destacam-se os analisadores preditivos e, em particular, os analisadores LL, em geral implementados manualmente como um conjunto de funções mutuamente recursivas, técnica conhecida como descida recursiva. Os métodos ascendentes, por sua vez, constroem a árvore das folhas para a raiz, agrupando progressivamente os *tokens* da entrada em estruturas maiores até alcançar o símbolo inicial; os analisadores LR, mais expressivos que os LL, costumam ser construídos por meio de ferramentas automáticas, como Yacc e Bison, devido à complexidade das tabelas que utilizam (AHO *et al.*, 2008). Em ambas as estratégias, a entrada é consumida da esquerda para a direita, um símbolo de cada vez.

Espera-se de um analisador sintático mais do que a simples aceitação ou rejeição da entrada. Ele deve sinalizar erros de forma clara e precisa, identificar o local de cada erro no programa fonte e, sempre que possível, recuperar-se de modo a permitir a detecção de erros subsequentes em uma única invocação do compilador (AHO *et al.*, 2008). Aho *et al.* (AHO *et al.*, 2008) discutem quatro estratégias gerais de recuperação de erros sintáticos. A recuperação em *modo pânico* descarta *tokens* da entrada até encontrar um *token* de sincronismo, como ponto-e-vírgula ou chave de fechamento, retomando a análise a partir desse ponto; é simples e robusta, embora possa ocultar erros próximos. A recuperação em nível de frase tenta corrigir o erro

localmente por meio de pequenas substituições, inserções ou exclusões de *tokens*. A estratégia de produções de erro estende a gramática com regras que reconhecem construções erradas comuns, permitindo emitir mensagens de erro especializadas. Por fim, a correção global busca a menor sequência de transformações capaz de tornar a entrada sintaticamente válida; embora teoricamente atraente, é dispendiosa demais para uso em compiladores reais.

Para os fins deste trabalho, a análise sintática é a fase responsável por reconhecer a estrutura gramatical da linguagem de domínio específico proposta, transformando a sequência de *tokens* produzida pela análise léxica em uma representação estruturada que descreve, de forma não ambígua, o autômato finito especificado pelo usuário. As decisões de projeto da gramática dessa linguagem, bem como o método de análise adotado em sua implementação, serão detalhados em capítulo posterior.

3.4 ANÁLISE SEMÂNTICA

A análise semântica é a terceira fase do *front-end* e tem por objetivo verificar a consistência do programa em relação a regras que não podem ser expressas pela gramática livre de contexto utilizada na fase sintática. Apoiando-se na árvore de derivação produzida pela análise sintática e nas informações armazenadas na tabela de símbolos, o analisador semântico identifica violações das regras de escopo, declaração e tipagem da linguagem fonte e, ao mesmo tempo, anota a árvore com informações adicionais, como tipos inferidos e conversões implícitas, que serão utilizadas pelas fases subsequentes de geração de código (AHO *et al.*, 2008).

A parcela mais visível do trabalho do analisador semântico é a **verificação de tipos**. Para realizá-la, o compilador atribui uma expressão de tipo a cada componente do programa fonte e verifica se essas expressões satisfazem o conjunto de regras lógicas que constitui o **sistema de tipos** da linguagem. Exige-se tipicamente, por exemplo, que o índice usado para acessar um arranjo seja inteiro, que os operandos de um operador aritmético tenham tipos compatíveis e que o valor retornado por uma função tenha o tipo declarado em sua assinatura (AHO *et al.*, 2008). Quando alguma dessas condições é violada, o compilador emite uma mensagem de erro de tipo, evitando que o programa avance para fases posteriores em estado inconsistente.

A verificação de tipos pode ser feita estaticamente, em tempo de compilação, ou dinamicamente, em tempo de execução. Um sistema de tipos é dito **seguro** quando a verificação estática realizada pelo compilador é suficiente para garantir que erros de tipo nunca ocorrerão durante a execução do programa objeto, dispensando, portanto, verificações dinâmicas dispendiosas; uma implementação de linguagem é dita **fortemente tipada** (*strongly typed*) quando o compilador efetivamente assegura essa propriedade para todos os programas que aceita (AHO *et al.*, 2008). A verificação

estática de tipos é, em geral, considerada uma das principais ferramentas para a detecção precoce de erros e tem aplicações que vão além do contexto restrito da compilação, como a verificação de *bytecodes* Java antes de sua execução em uma máquina virtual.

Aho et al. (AHO *et al.*, 2008) distinguem duas formas complementares de verificação de tipos. Na **síntese de tipos**, o tipo de uma expressão é construído a partir dos tipos de suas subexpressões, com base em regras locais; essa abordagem exige, em geral, que todos os nomes utilizados sejam declarados antes de seu uso, de modo que o compilador disponha sempre da informação necessária à composição dos tipos. Na **inferência de tipos**, por outro lado, o tipo de uma construção é deduzido a partir do modo como ela é utilizada, por meio de variáveis de tipo e regras de unificação; essa abordagem é típica de linguagens funcionais como ML e Haskell, que verificam os tipos sem exigir declarações explícitas. Em qualquer um dos dois modelos, o objetivo final é o mesmo, isto é, associar a cada expressão um tipo único e bem definido antes da geração de código.

Quando os operandos de uma operação possuem tipos diferentes, mas compatíveis em algum sentido, o compilador pode inserir automaticamente uma **conversão de tipo**, também denominada **coerção**, para uniformizar os operandos. Ao avaliar a soma de um inteiro com um número de ponto flutuante, por exemplo, é comum que o inteiro seja convertido para a representação de ponto flutuante antes da operação (AHO *et al.*, 2008). Essas conversões são denominadas implícitas quando inseridas pelo compilador sem intervenção do programador, e explícitas (ou *casts*) quando declaradas no próprio código fonte. As regras que regem cada tipo de conversão variam de linguagem para linguagem e integram a especificação semântica da linguagem fonte.

A verificação de tipos é, em geral, a parcela mais elaborada da análise semântica, mas não é a única. O analisador semântico também é responsável por verificar regras de escopo, garantindo que cada uso de um identificador corresponda a uma declaração visível no contexto em questão, por detectar declarações duplicadas em um mesmo escopo, por validar a unicidade de rótulos e por assegurar que estruturas como condicionais, laços e retornos respeitem as restrições impostas pela linguagem. Em linguagens com sobrecarga de funções ou operadores, cabe-lhe ainda escolher, entre as várias definições disponíveis para um mesmo nome, aquela cuja assinatura é compatível com o contexto de uso (AHO *et al.*, 2008).

Para os fins deste trabalho, a análise semântica desempenha um papel particular, pois a linguagem de domínio específico proposta descreve estruturas matemáticas (autômatos finitos) cuja correção depende de um conjunto de condições estruturais que vão além da sintaxe. Cabe à fase semântica verificar, por exemplo, se todos os estados referenciados em transições foram previamente declarados, se o estado inicial e cada estado final pertencem ao conjunto de estados, se cada símbolo utilizado em uma

transição pertence ao alfabeto especificado e, no caso de autômatos determinísticos, se não existem transições conflitantes a partir de um mesmo estado para um mesmo símbolo. Essas verificações, embora distintas da tipagem tradicional dos compiladores convencionais, seguem o mesmo princípio geral, isto é, identificar inconsistências estruturais antes que o autômato seja submetido à simulação descrita na Seção 2.7.

3.5 AST (ÁRVORE SINTÁTICA ABSTRATA)

A árvore de derivação produzida pela análise sintática, também chamada de **árvore sintática concreta**, espelha fielmente a aplicação das produções da gramática durante o reconhecimento da entrada. Por essa razão, ela costuma conter nós internos cuja função é puramente auxiliar, isto é, nós que existem apenas para resolver questões gramaticais como precedência, associatividade ou eliminação de recursão à esquerda, mas que não correspondem a nenhuma construção semanticamente significativa da linguagem fonte (AHO *et al.*, 2008). Para as fases posteriores do compilador, esses nós auxiliares representam ruído, pois aumentam o tamanho da árvore sem agregar informação útil sobre o programa.

A **árvore sintática abstrata** (*abstract syntax tree*, ou AST) é uma representação intermediária que elimina esse ruído, conservando apenas a estrutura essencial do programa fonte. Em uma AST de uma expressão, cada nó interno representa um operador ou uma construção da linguagem, e seus filhos representam os operandos ou componentes semanticamente significativos dessa construção (AHO *et al.*, 2008). Não há nós para variáveis sintáticas auxiliares nem para produções unitárias, e os terminais que servem apenas como pontuação (parênteses, ponto-e-vírgula, palavras-chave de delimitação) costumam ser descartados, pois sua função já está implícita na estrutura hierárquica da árvore. O resultado é uma estrutura mais compacta, mais próxima da intenção do programador e, por consequência, mais conveniente para ser percorrida pelas fases de análise semântica, otimização e geração de código.

A Figura 8 apresenta a AST da expressão $id + id * id$. Comparada à árvore de derivação da mesma expressão, exibida na Figura 7, observa-se a redução significativa do número de nós, pois desaparecem todos os nós internos rotulados com as variáveis auxiliares E , T e F , preservando-se apenas os operadores e os operandos efetivos. A precedência dos operadores, antes expressa pela profundidade dos nós auxiliares, agora é capturada diretamente pela própria estrutura da árvore.

A construção da AST costuma ser entrelaçada com a análise sintática, por meio das ações semânticas associadas a cada produção da gramática, conforme o esquema da tradução dirigida por sintaxe descrito na Seção 3.3. À medida que o analisador sintático reconhece uma produção, executa-se a ação correspondente, que tipicamente cria um nó da AST e o conecta aos nós já criados para os componentes da produção.

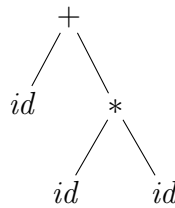


FIGURA 8 – Árvore sintática abstrata da expressão $id + id * id$

FONTE: Os autores (2026)

Ao término da análise, o ponteiro para a raiz da árvore representa toda a estrutura do programa fonte e é repassado às fases seguintes (AHO *et al.*, 2008).

Do ponto de vista de implementação, cada nó de uma AST é, em geral, um registro contendo um rótulo que identifica o tipo do nó (por exemplo, atribuição, soma, identificador, literal), um conjunto de ponteiros para os filhos e, eventualmente, atributos específicos do tipo de nó, como o nome de um identificador ou o valor de uma constante (AHO *et al.*, 2008). As fases subsequentes do compilador percorrem essa estrutura, lendo e anotando os nós conforme suas necessidades. A análise semântica, por exemplo, anota cada nó com seu tipo inferido e, quando necessário, insere nós adicionais representando conversões implícitas; o gerador de código intermediário, por sua vez, percorre a árvore anotada e emite, para cada nó, as instruções correspondentes na linguagem intermediária.

Para os fins deste trabalho, a AST representa o objeto central manipulado pela ferramenta proposta. Cada programa escrito na linguagem de domínio específico descreve um autômato finito, e sua AST organiza, em uma estrutura hierárquica única, os componentes desse autômato, ou seja, o conjunto de estados, o alfabeto, o conjunto de transições, o estado inicial e o conjunto de estados finais. É sobre essa estrutura, e não sobre o texto fonte, que operam tanto a análise semântica, descrita na Seção 3.4, quanto a etapa subsequente de simulação visual, conforme apresentada na Seção 2.7.

3.6 LINGUAGENS DE DOMÍNIO ESPECÍFICO (DSL)

As seções anteriores apresentaram os fundamentos do processo de compilação tomando como cenário implícito o de uma linguagem de programação de propósito geral, isto é, uma linguagem projetada para expressar qualquer programa computável, como C, Java ou Rust. Existe, contudo, uma classe distinta de linguagens, voltadas não para a generalidade, mas para a expressividade restrita a um domínio particular. Essa categoria recebe o nome de **linguagem de domínio específico** ou, na sigla consagrada em inglês, *Domain-Specific Language* (DSL). Esta seção apresenta o conceito de DSL, suas principais variantes e a relação que mantém com o ferramental de compiladores discutido nas seções precedentes.

Fowler (FOWLER; PARSONS, 2010) define uma DSL como uma linguagem

de programação de expressividade limitada, voltada para um domínio específico. A definição é deliberadamente curta, mas seus quatro elementos centrais merecem atenção. Em primeiro lugar, uma DSL é uma *linguagem de programação*, no sentido de que se destina a ser escrita e lida por seres humanos para expressar instruções a serem processadas por um computador, e não meramente um formato de dados. Em segundo lugar, é necessária a existência de uma *natureza linguística*, ou seja, os fragmentos isolados da DSL devem compor-se em construções maiores cujo sentido depende da forma como se articulam, e não apenas da soma das partes. Em terceiro lugar, sua *expressividade é limitada*, no sentido de que oferece apenas os recursos estritamente necessários para descrever o domínio que pretende cobrir, sem aspirar à generalidade computacional. Por fim, é *focada em um domínio*, e essa restrição de escopo é justamente o que torna a linguagem útil, pois permite que tanto a sintaxe quanto a semântica sejam moldadas em torno dos conceitos naturais daquele domínio.

A consequência prática dessa restrição é que uma DSL bem projetada permite que problemas do seu domínio sejam expressos de maneira mais concisa, mais legível e mais próxima da terminologia utilizada pelos especialistas do que seria possível em uma linguagem de propósito geral (FOWLER; PARSONS, 2010). Em contrapartida, uma DSL não se presta à escrita de programas fora do seu escopo. Não se trata de uma deficiência, mas de uma decisão de projeto, pois a perda de generalidade é o preço pago para obter um vocabulário e uma sintaxe mais alinhados ao problema que se quer resolver.

Fowler (FOWLER; PARSONS, 2010) distingue duas grandes categorias de DSLs conforme a estratégia de implementação adotada. Uma **DSL externa** é uma linguagem com sintaxe própria, independente da sintaxe de qualquer linguagem hospedeira, e cujo processamento exige a construção de um analisador, em geral por meio das técnicas clássicas de análise léxica e sintática descritas nas Seções 3.2 e 3.3. Exemplos consagrados desse tipo incluem expressões regulares, SQL, a linguagem de descrição de gráficos do Graphviz e os formatos de configuração estruturada com regras próprias. Uma **DSL interna**, por sua vez, é construída sobre uma linguagem hospedeira de propósito geral, aproveitando sua sintaxe e seu sistema de tipos para exprimir conceitos do domínio na forma de uma biblioteca, com nomes de funções e composições sintáticas escolhidos de modo a parecer uma linguagem própria. Linguagens como Ruby, Lisp, Scala e Kotlin são particularmente populares como anfitriãs de DSLs internas, devido à flexibilidade sintática que oferecem.

Cada uma das duas estratégias apresenta vantagens distintas. A DSL interna tem custo de implementação reduzido, pois reaproveita o compilador, o sistema de tipos e o ecossistema de ferramentas da linguagem hospedeira; em compensação, sua sintaxe está limitada por aquilo que a hospedeira permite expressar, e o usuário do domínio acaba exposto a detalhes da linguagem subjacente. A DSL externa tem o

custo inicial mais alto, uma vez que demanda a construção de um *front-end* dedicado, mas oferece liberdade total na escolha da sintaxe e isolamento completo em relação a qualquer linguagem hospedeira, o que costuma resultar em uma notação mais próxima do vocabulário do domínio (FOWLER; PARSONS, 2010).

Independentemente da categoria, Fowler (FOWLER; PARSONS, 2010) argumenta que toda DSL bem estruturada se organiza em torno de dois elementos. O primeiro é a **representação textual**, isto é, a sintaxe concreta que o usuário efetivamente lê e escreve. O segundo é o **modelo semântico**, uma estrutura de objetos, em geral construída em uma linguagem de propósito geral, que captura o significado do que foi escrito e sobre a qual se efetivamente operam as funcionalidades do sistema. O processamento de um programa escrito na DSL consiste, então, em traduzir a representação textual no correspondente modelo semântico, tarefa para a qual o ferramental de compiladores apresentado nas seções anteriores se aplica diretamente: a análise léxica converte caracteres em *tokens*, a análise sintática organiza os *tokens* em uma árvore, e fases subsequentes de tradução dirigida pela sintaxe constroem o modelo semântico a partir dessa árvore.

Essa visão é particularmente útil para situar o presente trabalho. A ferramenta proposta adota uma DSL externa, com sintaxe própria, voltada exclusivamente para a descrição declarativa de autômatos finitos. Seu modelo semântico é a representação interna do autômato, isto é, o conjunto de estados, o alfabeto, o conjunto de transições, o estado inicial e o conjunto de estados finais, organizados em uma estrutura de dados sobre a qual operam o verificador semântico e o simulador visual. As fases descritas nas Seções 3.2 a 3.4 correspondem precisamente ao processo de tradução da forma textual para esse modelo, e a árvore sintática abstrata, apresentada na Seção 3.5, é a estrutura intermediária por meio da qual essa tradução se concretiza.

3.7 CONSTRUÇÃO DE COMPILADORES (FERRAMENTAS)

A descrição apresentada nas seções anteriores dá conta de *o que* cada fase de um compilador realiza, mas é silenciosa sobre *como* essas fases são efetivamente implementadas em um sistema concreto. Construir manualmente, do zero, um analisador léxico, um analisador sintático e os mecanismos de tradução dirigida pela sintaxe é um esforço considerável e propenso a erros, sobretudo no caso de uma DSL externa, em que toda a infraestrutura precisa ser provida pelo próprio implementador. Por essa razão, ao longo das décadas consolidou-se um conjunto de ferramentas e bibliotecas voltadas justamente a automatizar partes substanciais desse trabalho. Esta seção apresenta as principais classes de ferramentas utilizadas na construção de compiladores, em particular daqueles destinados a DSLs, organizadas conforme a classificação proposta por Fowler (FOWLER; PARSONS, 2010).

A primeira e mais tradicional classe de ferramentas é a dos **geradores de analisadores sintáticos**, também chamados de *parser generators*. Nessa abordagem, o implementador descreve a linguagem por meio de uma gramática livre de contexto, escrita em uma notação próxima da BNF apresentada na Seção 3.3, eventualmente acrescida de ações semânticas associadas a cada produção. A ferramenta toma essa especificação como entrada e produz, automaticamente, o código fonte de um analisador completo, em geral em uma linguagem de propósito geral como C, Java ou Python (FOWLER; PARSONS, 2010). Pertencem a essa família ferramentas como Yacc e Bison, voltadas à análise ascendente do tipo LR, e ANTLR, particularmente popular para análise descendente preditiva. A vantagem central dessa abordagem é a correspondência direta entre a descrição formal da linguagem e o analisador gerado, o que torna o resultado fácil de revisar e de modificar quando a linguagem evolui. Em contrapartida, o uso desse tipo de ferramenta exige certa familiaridade com a teoria de gramáticas livres de contexto e com as limitações específicas da classe de *parsers* que a ferramenta produz, como as restrições LR, LL ou LALR.

Uma segunda classe de ferramentas é a dos **combinadores de parsers**, ou *parser combinators*. Em vez de gerar código a partir de uma gramática externa, essa abordagem fornece, na forma de uma biblioteca dentro de uma linguagem hospedeira, um conjunto de *parsers* elementares, capazes de reconhecer fragmentos triviais como um único caractere, uma palavra-chave ou um número, junto a um conjunto de combinadores que permitem construir *parsers* mais complexos a partir desses elementos por meio de composição (FOWLER; PARSONS, 2010). A gramática da linguagem aparece, então, no próprio código fonte do compilador, expressa diretamente pelas funções e operadores da biblioteca. Essa estratégia é particularmente comum em linguagens funcionais, como Haskell e Scala, mas também encontra expressão em linguagens modernas com bom suporte a tipos algébricos, sendo amplamente utilizada no ecossistema Rust por meio de bibliotecas como *nom* e *pest*. Sua principal vantagem é a ausência de qualquer etapa adicional de geração de código, o que integra perfeitamente o analisador ao restante do compilador, em uma única linguagem e em um único processo de compilação. A contrapartida é que o desempenho e a clareza dos diagnósticos podem depender de forma sensível da maneira como a gramática é estruturada, exigindo do implementador atenção a questões como retrocesso e ambiguidade.

Uma terceira opção, frequentemente subestimada, é a do **analisador escrito manualmente**, em geral pela técnica de descida recursiva, descrita na Seção 3.3. Nessa abordagem, cada não-terminal da gramática é traduzido em uma função do compilador, que invoca, direta ou indiretamente, as demais funções correspondentes aos não-terminais que aparecem em suas produções. Apesar de demandar mais trabalho inicial do que as duas alternativas anteriores, essa estratégia oferece o controle mais fino sobre o comportamento do analisador, sendo especialmente útil quando se

deseja produzir mensagens de erro de alta qualidade ou quando a linguagem inclui construções que se acomodam mal às restrições de um gerador automático (FOWLER; PARSONS, 2010). Não por acaso, muitos compiladores de produção, incluindo os de linguagens de propósito geral amplamente utilizadas, optam pela escrita manual do analisador.

Uma quarta classe, mais recente, é a das **language workbenches**, ou plataformas para engenharia de linguagens. Diferentemente das ferramentas anteriores, que se concentram quase exclusivamente na construção do *front-end*, uma *language workbench* oferece suporte integrado a todas as etapas de definição de uma DSL, da gramática à semântica, passando pelas ferramentas de edição, de verificação e até de geração de código (FOWLER; PARSONS, 2010). Sistemas como Xtext, JetBrains MPS e Spoofox permitem que o desenvolvedor da linguagem defina, em um único ambiente, a sintaxe, o sistema de tipos, os transformadores e os geradores, recebendo em troca um editor com realce de sintaxe, completamento automático e diagnóstico em tempo real, tudo derivado da própria definição da linguagem. Essa abordagem amplia consideravelmente a produtividade do projetista de DSLs, mas implica adesão a um ecossistema específico e introduz uma curva de aprendizado nada trivial.

A escolha entre essas alternativas depende de fatores como a complexidade da linguagem, o público-alvo, as exigências de desempenho do analisador e a familiaridade da equipe com as diferentes técnicas (FOWLER; PARSONS, 2010). Para os fins do presente trabalho, em que a DSL proposta é compacta, voltada exclusivamente à descrição de autômatos finitos, e cujo compilador integra um sistema mais amplo escrito em uma única linguagem hospedeira, a estratégia adotada contempla, de modo combinado, o uso de uma biblioteca de *parser combinators* para a construção do *front-end* e a escrita manual das demais etapas. Essa combinação preserva a integração natural com o restante da ferramenta, tal como apresentada no Capítulo 4, //vai ter um capítulo de rust? sem introduzir dependências de geradores externos ou de plataformas de *language engineering*.

3.8 DIAGNÓSTICO DE ERROS

A capacidade de relatar erros de forma precisa e útil é, ao lado da própria tradução, uma das duas funções essenciais de um compilador, conforme observado já na Seção 3.1. Em compiladores de linguagens de propósito geral, essa exigência costuma ser tratada como uma preocupação de engenharia entre outras, mas em compiladores de DSLs ela assume um peso ainda maior, pois o público que escreve programas em uma DSL inclui frequentemente especialistas do domínio sem formação extensa em programação (FOWLER; PARSONS, 2010). Para esses usuários, uma mensagem de erro confusa, fora de posição ou expressa em vocabulário técnico

estranho ao domínio pode inviabilizar o uso da linguagem com a mesma eficácia de um defeito de tradução. O presente trabalho, voltado a estudantes de teoria da computação e dirigido também a usuários em fase de aprendizagem, enquadra-se justamente nesse cenário, e por isso o diagnóstico de erros recebe aqui tratamento próprio.

Os erros que um compilador pode detectar distribuem-se ao longo das três fases do *front-end*, conforme já exposto nas seções correspondentes. A análise léxica detecta erros de formação de *tokens*, como caracteres isolados que não pertencem ao alfabeto da linguagem ou literais malformados. A análise sintática detecta erros de estrutura, isto é, sequências de *tokens* que, embora individualmente válidas, não compõem uma derivação admitida pela gramática, como a ausência de um delimitador esperado ou o uso de uma construção fora de contexto. A análise semântica detecta, por fim, erros que escapam à descrição puramente sintática, tais como o uso de identificadores não declarados, incompatibilidades de tipo ou violações de restrições próprias do domínio que a linguagem modela. Cada fase, portanto, contribui com um vocabulário próprio de erros, e cada uma demanda mecanismos específicos para detectá-los e para recuperar-se deles a fim de prosseguir o processamento.

Independentemente da fase em que ocorrem, os erros relatados por um compilador são tanto mais úteis quanto melhor satisfazem três critérios fundamentais. O primeiro é a **precisão da localização**, isto é, a indicação inequívoca, em termos de linha e coluna do arquivo fonte, do ponto do programa em que o erro foi detectado. O segundo é a **clareza da mensagem**, ou seja, a expressão do erro em uma linguagem natural compreensível para o usuário, evitando jargão técnico desnecessário e, sempre que possível, indicando o que se esperava encontrar e o que foi de fato encontrado. O terceiro é a **capacidade de detectar múltiplos erros em uma única execução**, evitando que o usuário tenha de repetir o ciclo de compilação diversas vezes para descobrir, sucessivamente, defeitos que poderiam ter sido reportados em conjunto.

A recuperação após o primeiro erro, requisito para que o compilador prossiga a análise e detecte os erros remanescentes, depende das estratégias já discutidas na Seção 3.3. Para erros sintáticos, o modo pânico oferece uma recuperação simples e robusta, ao custo de ignorar trechos do programa fonte; a recuperação em nível de frase tenta inserir, remover ou substituir *tokens* para reconstruir uma forma sintaticamente válida, com riscos controlados de provocar erros em cascata; e as produções de erro permitem que o projetista da linguagem antecipe construções incorretas frequentes e atribua a elas mensagens específicas. Para erros semânticos, a recuperação é, em geral, mais simples, pois a estrutura do programa já foi reconhecida pela fase sintática, restando apenas anotar o erro detectado e prosseguir a verificação dos demais nós da árvore.

No contexto de DSLs, Fowler (FOWLER; PARSONS, 2010) observa que a qualidade do diagnóstico está intimamente ligada à escolha da estratégia de implementação

discutida na Seção 3.7. Geradores automáticos de *parsers* costumam produzir mensagens genéricas, expressas em termos de *tokens* e estados internos do analisador, que pouco ajudam o usuário final. Combinadores de *parsers* permitem maior controle sobre as mensagens, mas requerem atenção do implementador para que esse controle seja efetivamente exercido. Analisadores escritos manualmente, por sua vez, oferecem o mais alto grau de personalização das mensagens, sendo essa, conforme já apontado, uma das principais razões pelas quais compiladores de produção continuam a ser escritos dessa maneira. *Language workbenches* costumam ir além do diagnóstico em *batch* e oferecer detecção de erros incremental, em tempo de edição, diretamente na interface gráfica do editor.

Para a ferramenta proposta neste trabalho, o diagnóstico de erros é tratado como parte central do projeto da DSL, e não como decoração posterior. Cada fase do compilador, da análise léxica à análise semântica, é instrumentada para registrar os erros encontrados em uma estrutura comum, preservando a localização exata no texto fonte e uma mensagem redigida em vocabulário próximo ao domínio dos autômatos finitos, com referências aos conceitos do Capítulo 2, como estados, transições, alfabeto e estados finais. Após o término de cada fase, esses erros são apresentados em conjunto ao usuário, e somente quando uma fase produz erros que impeçam a continuação do processamento é que o restante do compilador deixa de ser executado. Esse tratamento dá ao usuário uma visão panorâmica dos defeitos do programa em cada execução, em linha com os critérios de qualidade enumerados anteriormente, e prepara o terreno para os recursos visuais de apoio descritos no Capítulo ??.

3.9 DSL NO ENSINO

As seções anteriores deste capítulo apresentaram as DSLs como ferramentas voltadas a tornar mais concisa e expressiva a descrição de soluções em um domínio particular. Quando esse domínio é constituído pelos próprios objetos de estudo de uma disciplina acadêmica, a DSL passa a desempenhar, além do papel técnico habitual, uma função pedagógica, isto é, a de servir como linguagem por meio da qual o estudante exerce, formaliza e verifica os conceitos que aprende. Esta seção examina, com base no trabalho de Morazán e Antunez (MORAZÁN; ANTUNEZ, 2014), o uso de DSLs no ensino de linguagens formais e teoria da computação, situando essa prática como contexto imediato do presente trabalho.

O ensino tradicional dos conteúdos discutidos no Capítulo 2 costuma apoiar-se em ferramentas gráficas que permitem ao estudante construir autômatos e gramáticas manipulando diretamente seus elementos visuais, dos quais o exemplo mais difundido é o JFLAP (RODGER; FINLEY, 2006). Esse tipo de abordagem oferece, sem dúvida, ganhos importantes, em especial a redução do esforço inicial necessário para que o

estudante visualize um autômato e acompanhe sua execução. No entanto, conforme apontam Morazán e Antunez (MORAZÁN; ANTUNEZ, 2014), a ênfase exclusiva em ferramentas gráficas tende a separar o estudo de linguagens formais do restante das atividades de programação que o estudante realiza ao longo do curso, tratando o autômato como um diagrama a ser desenhado, e não como um objeto computacional a ser construído e manipulado por meio de código.

A alternativa proposta por Morazán e Antunez (MORAZÁN; ANTUNEZ, 2014) consiste em oferecer ao estudante uma DSL textual em que autômatos finitos, autômatos de pilha, máquinas de Turing e gramáticas sejam descritos diretamente como termos da linguagem, podendo então ser construídos, compostos, testados e executados como qualquer outro programa. Os autores argumentam que essa abordagem recoloca os objetos da teoria da computação no mesmo plano dos demais artefatos com que o estudante trabalha em programação, integrando o estudo das linguagens formais ao restante do currículo de computação em vez de tratá-lo como uma disciplina insulada com ferramental próprio.

Entre os ganhos pedagógicos dessa abordagem, os autores destacam a possibilidade de o estudante escrever, ele mesmo, programas que constroem autômatos a partir de outras estruturas, que verificam propriedades de máquinas dadas ou que comparam computações em diferentes formalismos, exercícios praticamente inviáveis em um ambiente exclusivamente gráfico (MORAZÁN; ANTUNEZ, 2014). Outro ganho relevante é a possibilidade de submeter os autômatos descritos na DSL a testes automatizados, no estilo dos testes unitários usuais em desenvolvimento de software, o que introduz, já no contexto da disciplina de linguagens formais, hábitos de verificação que serão úteis ao estudante em todo o restante de sua formação.

A abordagem textual não se opõe, entretanto, à visualização. Pelo contrário, uma DSL voltada ao ensino tende a beneficiar-se da combinação entre uma representação textual primária, em que o estudante exprime suas ideias, e uma representação visual derivada, gerada automaticamente a partir do texto, que serve para a inspeção do resultado e para a execução passo a passo da máquina descrita. Esse arranjo preserva os ganhos de programação enumerados anteriormente sem abrir mão dos ganhos visuais que motivam o uso de ferramentas como o JFLAP, e é precisamente esse o equilíbrio que a ferramenta proposta no presente trabalho busca alcançar, conforme detalhado nos capítulos subsequentes.

4 RUST COMO LINGUAGEM DE IMPLEMENTAÇÃO

Este capítulo apresenta a linguagem de programação Rust e justifica sua adoção como plataforma de implementação da ferramenta proposta. Inicia-se com uma visão geral da linguagem e de suas características centrais (Seção 4.1), seguindo-se uma discussão sobre seu sistema de tipos e o modelo de propriedade de memória (Seção 4.2). Em seguida, apresenta-se o ecossistema de bibliotecas e o gerenciador de pacotes Cargo (Seção 4.3), e discutem-se as principais bibliotecas de *parser combinators* disponíveis para Rust (Seção 4.4), cujas características orientam a escolha de implementação descrita no Capítulo ??.

4.1 VISÃO GERAL DA LINGUAGEM

Rust é uma linguagem de programação de sistemas criada pela Mozilla Research e lançada em sua primeira versão estável em 2015 (KLABNIK; NICHOLS, 2019). Seu projeto parte de uma premissa central: oferecer desempenho comparável ao de C e C++, sem abrir mão das garantias de segurança de memória que essas linguagens historicamente não fornecem em tempo de compilação. Para atingir esse objetivo, Rust introduz um sistema de tipos estático e um conjunto de regras de compilação, coletivamente denominado modelo de *ownership* e *borrowing*, que permite ao compilador verificar, antes da execução, a ausência de erros como acessos a regiões de memória inválidas, condições de corrida e uso de ponteiros nulos (KLABNIK; NICHOLS, 2019).

A linguagem é compilada para código nativo por meio do compilador `rustc`, que utiliza como *backend* a infraestrutura LLVM, a mesma empregada por compiladores como Clang e Swift (KLABNIK; NICHOLS, 2019). O resultado é um binário executável diretamente pelo sistema operacional, sem a necessidade de uma máquina virtual ou de um interpretador em tempo de execução. Essa característica torna Rust adequado tanto para o desenvolvimento de sistemas de baixo nível, como kernels e drivers, quanto para aplicações de alto nível que exigem desempenho previsível, como compiladores, servidores e ferramentas de linha de comando.

Do ponto de vista da expressividade, Rust combina paradigmas de programação imperativa, funcional e orientada a objetos por meio de *traits*, que funcionam de forma análoga às interfaces de Java ou às classes de tipos de Haskell (KLABNIK; NICHOLS, 2019). Essa flexibilidade permite que construções comuns em linguagens funcionais, como mapeamento, filtragem e encadeamento de iteradores, sejam expressas de forma concisa e com custo de abstração zero, isto é, sem sobrecarga de desempenho em relação ao código imperativo equivalente. Para a construção de compiladores e processadores de linguagens, esse conjunto de características é particularmente

valioso: a segurança de tipos facilita a representação de estruturas como tokens, nós de AST e estados de autômatos sem o risco de inconsistências em tempo de execução, enquanto o modelo funcional favorece a composição das fases de análise de forma clara e modular.

4.2 SISTEMA DE TIPOS, *OWNERSHIP* E *BORROWING*

O traço mais distintivo de Rust em relação a outras linguagens de sistemas é seu modelo de gerenciamento de memória, que elimina a necessidade de um coletor de lixo (*garbage collector*) sem exigir que o programador gerencie a memória manualmente (KLABNIK; NICHOLS, 2019). Esse modelo é fundamentado em três conceitos inter-relacionados: *ownership* (propriedade), *borrowing* (empréstimo) e *lifetimes* (tempos de vida).

O princípio de *ownership* estabelece que cada valor em Rust possui exatamente um **dono** em qualquer ponto do programa, e que o valor é automaticamente liberado da memória quando seu dono sai de escopo (KLABNIK; NICHOLS, 2019). A atribuição de um valor a uma nova variável, ou sua passagem como argumento a uma função, transfere a propriedade para o novo contexto, invalidando o acesso anterior. Essa regra, verificada estaticamente pelo compilador, garante que não existam dois fragmentos de código tentando liberar a mesma região de memória simultaneamente.

O *borrowing* complementa o *ownership* ao permitir que um valor seja **referenciado** sem que sua propriedade seja transferida. Rust distingue dois tipos de referências: as **referências imutáveis** (`&T`), das quais podem existir simultaneamente quantas forem necessárias, e as **referências mutáveis** (`&mut T`), das quais apenas uma pode existir em qualquer instante, excluindo a coexistência com qualquer referência imutável (KLABNIK; NICHOLS, 2019). Essas restrições, verificadas pelo compilador em uma fase denominada *borrow checking*, eliminam em tempo de compilação toda uma classe de erros relacionados à mutação concorrente de dados compartilhados.

O sistema de tipos de Rust é estático e fortemente tipado, com suporte a inferência de tipos que dispensa, na maior parte dos casos, anotações explícitas de tipo (KLABNIK; NICHOLS, 2019). Para a representação de estruturas de dados que podem assumir diferentes formas, Rust oferece os **tipos enumerados** (`enum`), que permitem associar dados distintos a cada variante e são processados por meio de correspondência de padrões (*pattern matching*) com verificação de exaustividade pelo compilador. Esse recurso é particularmente útil na implementação de compiladores, onde nós de AST, tokens e erros costumam ser representados naturalmente como enumerações: o compilador garante que todo código que manipula essas estruturas trate explicitamente cada variante possível, evitando casos não tratados silenciosos.

No contexto do presente trabalho, o sistema de tipos e o modelo de *ownership*

de Rust oferecem garantias concretas para a implementação das fases do compilador da DSL proposta. A AST do autômato, por exemplo, pode ser representada como uma enumeração cujas variantes correspondem aos diferentes nós da árvore, estados, transições, declarações de alfabeto e metadados, com a certeza de que qualquer acesso a essas estruturas será verificado em tempo de compilação. Da mesma forma, a passagem da AST entre as fases de análise semântica e de simulação não requer cópias desnecessárias, pois o modelo de *borrowing* permite que cada fase leia a estrutura sem assumir sua propriedade.

4.3 ECOSSISTEMA E GERENCIADOR DE PACOTES CARGO

O ecossistema de Rust organiza-se em torno do **Cargo**, ferramenta oficial que acumula as funções de gerenciador de pacotes, sistema de *build* e executor de testes (KLABNIK; NICHOLS, 2019). Por meio do Cargo, o desenvolvedor declara as dependências do projeto em um arquivo de manifesto (`Cargo.toml`), e a ferramenta resolve automaticamente as versões compatíveis, baixa os pacotes do repositório central `crates.io` e compila todo o projeto em uma única invocação. Esse modelo elimina a necessidade de sistemas de *build* externos e garante que projetos Rust sejam reproduzíveis em qualquer ambiente que disponha do compilador instalado.

A unidade de distribuição de código em Rust é denominada **crate**, termo que corresponde, aproximadamente, ao conceito de biblioteca ou módulo em outras linguagens (KLABNIK; NICHOLS, 2019). O repositório `crates.io` hospeda dezenas de milhares de *crates* de código aberto, cobrindo desde estruturas de dados e algoritmos até bibliotecas para desenvolvimento *web*, interfaces gráficas e, em especial para os fins deste trabalho, análise sintática e processamento de linguagens.

A integração entre Cargo e o compilador `rustc` estende-se à execução automatizada de testes unitários e de integração. Todo projeto Rust pode incluir módulos de teste diretamente no código fonte, anotados com o atributo `#[test]`, que são compilados e executados pelo comando `cargo test` (KLABNIK; NICHOLS, 2019). Esse mecanismo nativo de testes é relevante para o presente trabalho, pois permite validar cada fase do compilador da DSL de forma isolada, verificando o comportamento do analisador léxico, do analisador sintático e do verificador semântico sobre entradas controladas, em linha com a estratégia de validação descrita no Capítulo ??.

4.4 PARSER COMBINATORS EM RUST

Conforme discutido na Seção 3.7, a estratégia de implementação adotada para o compilador da DSL proposta baseia-se no uso de uma biblioteca de *parser combinators*, abordagem que integra naturalmente a definição da gramática ao restante do código da ferramenta. O ecossistema Rust oferece três bibliotecas principais para esse fim: **nom**,

pest e **chumsky**, cada uma com características distintas que as tornam adequadas a diferentes cenários.

A biblioteca **nom** é uma das mais antigas e consolidadas do ecossistema Rust para análise sintática (COUPRIE, 2015). Sua abordagem é baseada em funções que recebem uma fatia de bytes ou caracteres e retornam o resultado do reconhecimento juntamente com o restante da entrada não consumida, compondo-se por meio de macros e funções de ordem superior. A orientação de **nom** para o processamento de formatos binários e protocolos de rede, domínio em que a eficiência é crítica, torna-a uma escolha robusta para linguagens textuais, embora a geração de mensagens de erro precisas requeira esforço adicional do implementador, pois a biblioteca não foi projetada com esse objetivo em primeiro plano.

A biblioteca **pest** adota uma abordagem diferente: em vez de compor *parsers* no código Rust, o desenvolvedor descreve a gramática da linguagem em um arquivo externo com notação própria, derivada de gramáticas de expressão de análise (*Parsing Expression Grammars*, ou PEG), e a biblioteca gera automaticamente o analisador correspondente em tempo de compilação por meio de macros procedurais (FLORESCU; CONTRIBUTORS, 2018). Essa estratégia separa claramente a especificação da gramática de sua implementação, facilitando a leitura e a manutenção da definição da linguagem, mas reduz o controle sobre o comportamento do analisador e sobre as mensagens de erro produzidas.

A biblioteca **chumsky** é a mais recente das três e foi projetada explicitamente com foco na qualidade do diagnóstico de erros (WALTON; CONTRIBUTORS, 2022). Seus combinadores propagam automaticamente informações de localização ao longo do processo de análise e oferecem mecanismos nativos de recuperação de erros, permitindo que o analisador continue após encontrar uma construção inválida e reporte múltiplos erros em uma única execução. Essas características alinham-se diretamente com os requisitos de diagnóstico estabelecidos na Seção 3.8, tornando **chumsky** uma alternativa particularmente adequada para DSLs voltadas ao ensino, em que a qualidade das mensagens de erro é tão importante quanto a correção da análise.

O Quadro 3 sintetiza as principais características das três bibliotecas, evidenciando os critérios mais relevantes para a escolha a ser realizada durante a implementação descrita no Capítulo ??.

A escolha entre as três bibliotecas será orientada, durante a fase de implementação, pelos requisitos estabelecidos para o compilador da DSL: em especial, a necessidade de mensagens de erro com indicação precisa de localização e natureza do problema, conforme descrito na Seção 3.8, e a integração natural com o restante da ferramenta, escrita integralmente em Rust. A análise comparativa apresentada nesta seção fornece o embasamento necessário para essa decisão, que será documentada e justificada no Capítulo ??.

TABELA 3 – Comparação entre bibliotecas de *parser combinators* para Rust

Critério	nom	pest	chumsky
Abordagem	Combinadores	Gramática PEG externa	Combinadores
Qualidade dos erros	Baixa	Média	Alta
Recuperação de erros	Manual	Limitada	Nativa
Rastreamento de localização	Manual	Automático	Automático
Maturidade	Alta	Alta	Média
Foco original	Dados binários	Linguagens textuais	Linguagens textuais

FONTE: Os autores (2026)

REFERÊNCIAS

- AHO, A. V. *et al.* **Compiladores: Princípios, Técnicas e Ferramentas**. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.
- COUPRIE, G. **nom: Rust parser combinators framework**. 2015. <https://github.com/rust-bakery/nom>.
- DIVERIO, T. A.; MENEZES, P. B. **Teoria da Computação: Máquinas Universais e Computabilidade**. 3. ed. Porto Alegre: Bookman, 2011.
- FLORESCU, D.; CONTRIBUTORS. **pest: The Elegant Parser**. 2018. <https://pest.rs>.
- FOWLER, M.; PARSONS, R. **Domain-Specific Languages**. Upper Saddle River, NJ: Addison-Wesley Professional, 2010. (Addison-Wesley Signature Series).
- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. **Introduction to Automata Theory, Languages, and Computation**. 3. ed. Boston, MA: Pearson Addison-Wesley, 2006.
- KLABNIK, S.; NICHOLS, C. **The Rust Programming Language**. San Francisco: No Starch Press, 2019.
- LEWIS, H. R.; PAPADIMITRIOU, C. H. **Elements of the Theory of Computation**. 2. ed. Upper Saddle River, NJ: Prentice Hall, 1998.
- MORAZÁN, M. T.; ANTUNEZ, R. Functional automata: Formal languages for computer science students. In: **Proceedings of the 3rd International Workshop on Trends in Functional Programming in Education (TFPIE 2014)**. [S.l.]: Open Publishing Association, 2014. (Electronic Proceedings in Theoretical Computer Science, v. 170), p. 19–32.
- RODGER, S. H.; FINLEY, T. W. **JFLAP: An Interactive Formal Languages and Automata Package**. Sudbury, MA: Jones and Bartlett Publishers, 2006.
- SIPSER, M. **Introduction to the Theory of Computation**. 3. ed. Boston, MA: Cengage Learning, 2012.
- VIEIRA, N. J. **Introdução aos Fundamentos da Computação: Linguagens e Máquinas**. São Paulo: Thomson Learning, 2006.
- WALTON, J.; CONTRIBUTORS. **chumsky: A parser library for humans with powerful error recovery**. 2022. <https://github.com/zesterer/chumsky>.